How Every Test Was Done

A step-by-step guide to understanding, explaining and re-doing each test yourself

Mihir Bhati (24BCE5341) · Ahana Sharan (24BCE5334)

25 October 2026 · Project folder: D:\MyClaudeProject\nlp

Why this guide exists. Your teacher allows AI use, but expects you to understand what was done and to be able to do any step yourself. Claude Code wrote the code and ran the tests. This guide teaches the recipe behind every test, gives a small example you can work out on paper, and the exact command to run it yourself, with what your screen should show.
How to use it: read Part A first, slowly: it shows where every file is and how a score is calculated from them. Then do the 30-minute rehearsal in Part E once before you present.

Contents

1. Part A. The basics: what a test is, where the files are, and how a score is calculated
2. Part B. Software tests: writing and running your own
3. Part C. Every experiment, one by one (15 tests)
4. Part D. If the teacher says "do it now"
5. Part E. 30-minute rehearsal checklist
6. Part F. How to explain the AI use honestly
7. Part G. Build it yourself from scratch: a short script for every experiment

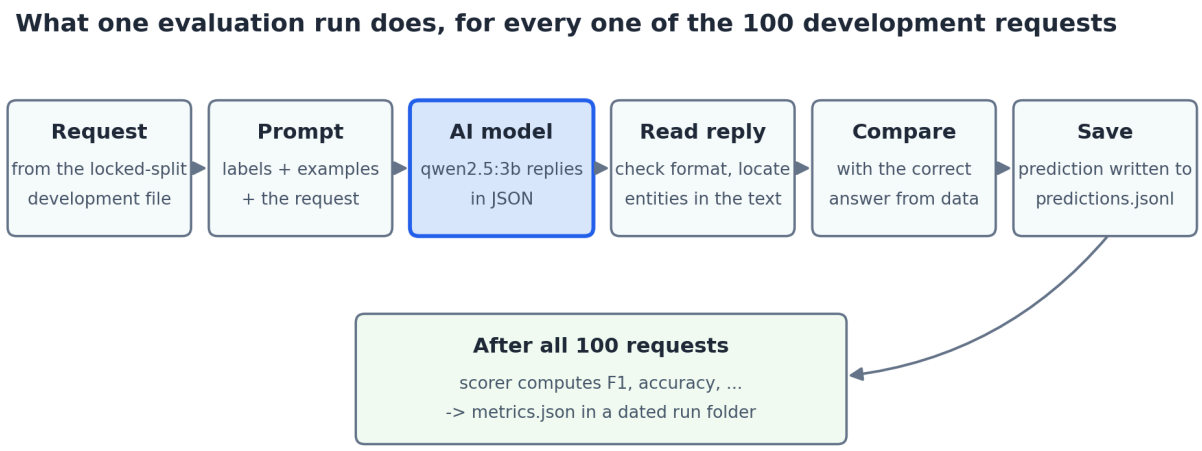
Part A. The basics

A1. There are two kinds of "test" in this project

	Software tests	Experiments (evaluations)
Question	Does the code do what it should?	How good is each system at the task?
Example	"If every answer is right, the scorer must give F1 = 1.0"	"How many of 100 real requests does the agent understand?"
Answer	pass or fail	a score, such as F1 = 0.974
How many	371	over 60 recorded runs
Run with	python -m pytest tests	the scripts in the scripts folder
Where it lives	tests folder	results in the runs folder

Both matter. Software tests prove the measuring tools work; experiments use those tools to measure the systems. A score from a broken scorer would mean nothing, which is why the scorer itself has tests.

A2. The recipe every experiment follows



Every experiment repeats these steps for each of the 100 development requests.

1. **Take a request** from the development file, for example "make a room darker". The dataset also gives its correct answer: intent `iot_hue_lightdim`, entity `house_place = room`.
2. **Build the prompt**: the list of allowed intents, a few worked examples, then the request.
3. **Ask the AI model.** It replies in JSON, e.g. `{"intent": "iot_hue_lightdim", "entities": [...]}`.
4. **Read the reply**: check the format is valid and find each extracted entity in the request text.
5. **Compare** with the correct answer: right or wrong.
6. **Save** the prediction, then move to the next request.
7. **After all 100**, the scorer counts rights and wrongs and computes F1, accuracy and the other scores, and saves them in a dated run folder.

That is the whole idea. The different experiments change only **which system** answers (B0, B1, A1, A2...) or **which requests** it gets (clean, with typos...).

A3. Where everything lives: the files behind every test

Before running anything, it helps to know that the whole project is just **folders of files** on your laptop, in `D:\MyClaudeProject\nlp`. Open that folder in File Explorer and you will see these:

Folder	What is inside	You will use it for
data\processed\P1\	The requests and their correct answers for project P1	seeing the data the systems are tested on
scripts\	Small programs you run, one per job (walkthrough.py, explain_f1.py, demo.py...)	running every test
src\	The framework itself: the code the scripts use	seeing how a step works
runs\	One folder per experiment, holding its answers and its scores	checking any result
tests\	The 371 automated tests	Part B
reports\	Reports, charts and the files these tools save	showing results

The two files every test is built from

File 1: the correct answers. `data\processed\P1\dev.jsonl` holds the 100 development requests for P1 and, for each one, the correct answer. These come from the MASSIVE dataset, where people labelled every request. Open it with Notepad (right-click → Open with → Notepad). Each line is one request. The second line is:

```
{"case_id": "massive-24", "utt": "make a room darker", "true_intent": "iot_hue_lightdim",
 "in_scope": true, "entities": [{"type": "house_place", "text": "room", "start": 7, "end":
11}], ...}
```

Field	Meaning	In this line
case_id	the request's ID in the MASSIVE dataset	massive-24
utt	the request, exactly as a user typed it (utterance)	make a room darker
true_intent	the correct intent	iot_hue_lightdim (dim a light)
in_scope	true = the app should handle it; false = off-topic, should be refused	true
entities	the details to extract, with their position in the sentence (counting characters from 0)	house_place = "room", characters 7 to 11

The format is called **JSONL**: one record per line, written as `"name": value` pairs.

File 2: what a system answered. Every experiment saves its answers in its own folder in `runs`. For A2 on P1 that is `runs\20261024_1654_P1_A2-selected_0\predictions.jsonl`. The folder name is the date and time (24 Oct, 16:54), the project (P1) and the system (A2). The line for the same request is:

```
{"case_id": "massive-24", "gold_intent": "iot_hue_lightdim", "gold_in_scope": true, "status":
"ok",
 "predicted_intent": "iot_hue_lightdim", "predicted_entities": [{"type": "house_place",
"text": "room"}], ...}
```

Field	Meaning
gold_intent	the correct intent, copied from File 1 ("gold" means correct answer)
status	ok = the system answered with an intent; oos = it refused the request as off-topic; invalid or error = its reply was broken
predicted_intent	the intent the system answered
predicted_entities	the details the system extracted

Every score in this project is just these two files compared line by line. Request massive-24: correct `iot_hue_lightdim`, answered `iot_hue_lightdim`, so it is right. Section A5 shows how all 80 lines become an F1 score.

A4. The walkthrough script, explained

A **script** is a text file of instructions written in Python. `walkthrough.py` is one of them, in the scripts folder: `D:\MyClaudeProject\nlp\scripts\walkthrough.py`. You can open it to read it, in Notepad or in VS Code (installed on this laptop), with:

```
code scripts\walkthrough.py
```

It is a teaching tool: it does what a real experiment does, but for only three requests, and prints every step instead of doing it silently. It changes nothing and saves nothing.

What it does, step by step, and where each step's code is

Step	What happens	Files it reads	Code that does it
Start	Locks the test data, then reads the project's intents, fields and description	data\manifests\P1.json, configs\descriptions\P1.md	src\analyzer\spec.py, src\analyzer\access.py
1	Takes a request and its correct answer	data\processed\P1\dev.jsonl (File 1)	load_cases in src\analyzer\spec.py
2	Picks example requests to show the model (fixed, or the most similar ones)	data\processed\P1\kshot.jsonl (120 labelled examples)	src\pipelines\examples.py
3	Writes the prompt: instructions, the list of intents, the examples, then the request	—	render_prompt in src\pipelines\catalogue.py
4	Sends the prompt to the AI model and receives its reply	cache\ folder (the model's saved answers), or the live model with --live	src\llm\client.py, which talks to Ollama at http://127.0.0.1:11434
5	Reads the reply: is it valid JSON with a real intent? Where is each entity in the sentence?	—	src\evaluation\output_schema.py, src\evaluation\span_rule.py
6	Compares the answer with the correct one: right or wrong	—	inside walkthrough.py
7	Scores the requests, with our scorer and with scikit-learn	—	src\evaluation\scorer.py and the scikit-learn library

"From the saved cache" means: when the experiments ran, every answer the model gave was saved in the `cache` folder. The walkthrough reuses those saved answers, so you see exactly what the experiments saw, instantly. Add `--live` to ask the model fresh; because the model is repeatable, it gives the same answers.

How to run it

1. Open the **Ollama** app from the Start menu (it runs the AI model in the background).
2. Open **PowerShell** (Start → type PowerShell → Enter).
3. Go to the project folder by typing this and pressing Enter:

```
cd D:\MyClaudeProject\nlp
```

1. Run the script:

```
python scripts/walkthrough.py
```

It prints three requests: two the app should handle and one off-topic one. For each, steps 1 to 6 appear under it. Here is request 2, with what each part means:

```
> python scripts/walkthrough.py   (request 2 of 3 shown)

REQUEST 2: "make a room darker"

  1. Correct answer (from the dataset):
    intent = iot_hue_lightdim
    entities = house_place='room'

  2. Examples put in the prompt (12 in total, first 3 shown):
    "review the alarms i have" -> alarm_query
    "remove alarm of seven thirty am on monday" -> alarm_remove
    "put alarm on" -> alarm_set

  3. Prompt sent to the model (68 lines, start and end shown):
    Label this request and extract its information. Reply with JSON only.

    Supported requests:
    - alarm_query
    - alarm_remove
    - alarm_set
       ... (58 more lines: label list, examples) ...
    Use an empty list if there is nothing to extract. Never invent a value.

    Request: make a room darker
    Answer:

  4. Model's raw reply:
    {"intent": "iot_hue_lightdim", "entities": [{"type": "house_place", "text": "room"}]}

  5. Reading the reply:
    valid format = True
    status = ok   intent = iot_hue_lightdim
    entities located in the request text = house_place='room' at characters 7-11

  6. Verdict:
    intent RIGHT
```

Request 2 of 3 (real output).

What you see	What it means
1. Correct answer	read from File 1: the dataset says this request is iot_hue_lightdim, with house_place = room
2. Examples put in the prompt	worked examples the model is shown first, so it learns the format (12 here: one per intent)
3. Prompt sent to the model	the full text the model receives: 68 lines, of which the start and end are shown
4. Model's raw reply	exactly what the model wrote back, as JSON
5. Reading the reply	the format is valid, the intent is a real one, and "room" is found at characters 7 to 11 of the request
6. Verdict	the answer matches the correct one, so it is right

```
> python scripts/walkthrough.py   (end of the output)

7. Scores for these 3 requests
   our scorer:   intent accuracy 1.000, entity F1 1.000, off-topic F1 1.000
   scikit-learn: intent accuracy 1.000
   A real evaluation does exactly this for all 100 development requests, saves every
   prediction in a run folder, and computes macro-F1 over all intents.
```

The end of the output: all three right, so every score is 1.000.

How this relates to the real experiments: an experiment runs exactly these steps, with the same code, for all 100 development requests instead of 3. It saves every answer in a predictions.jsonl file (File 2) and the scores in metrics.json, inside a new run folder.

Other things to try:

```
python scripts/walkthrough.py --n 5                    # 5 requests instead of 3
python scripts/walkthrough.py --design schema-sim-llm  # the design A2 chose
python scripts/walkthrough.py --live                   # ask the model fresh
```

A5. Work out F1 yourself, on the real results

A2's recorded intent macro-F1 on P1 is **0.974**. Here is how to get that number yourself from Files 1 and 2.

Option 1: let the explain script show every step

```
python scripts/explain_f1.py
```

The script `scripts\explain_f1.py` reads File 1 and File 2, puts them side by side, does the counting, and prints each step. It also saves the side-by-side table as a spreadsheet file.

```
> python scripts/explain_f1.py   (steps 1 to 3)

========================================================================
STEP 1. Correct answers:  data\processed\P1\dev.jsonl  (100 requests)
STEP 2. System's answers: runs\20261024_1654_P1_A2-selected_0\predictions.jsonl
        80 requests are in scope (they have a real intent); 20 are off-topic and are scored
        separately (off-topic F1), so intent F1 uses only the in-scope ones.
========================================================================

STEP 3. Side by side (first 15 of 80; all of them are in the CSV file)

    #  request                              correct intent       system said            right?
    1  turn the lights off please           iot_hue_lightoff     iot_hue_lightoff       yes
    2  make a room darker                   iot_hue_lightdim     iot_hue_lightdim       yes
    3  set an alarm at six in the morning   alarm_set            alarm_set              yes
    4  it's too dark here                   iot_hue_lighton      iot_hue_lighton        yes
    5  turn off the smart plug              iot_wemo_off         iot_wemo_off           yes
    6  set a wake up call for ten am        alarm_set            alarm_set              yes
    7  set an alarm for one p. m.           alarm_set            alarm_set              yes
    8  i want to wake up at six am tomorrow alarm_set            alarm_set              yes
    9  turn on the t. v. socket when i arriv iot_wemo_on          iot_wemo_on            yes
   10  can you turn off wemo plug socket    iot_wemo_off         iot_wemo_off           yes
   11  make coffee now                      iot_coffee           iot_coffee             yes
   12  at one p. m. today start the robot va iot_cleaning        iot_cleaning           yes
   13  olly start the vacuum cleaner        iot_cleaning         iot_cleaning           yes
   14  can you start the roomba             iot_cleaning         iot_cleaning           yes
   15  i need some more lights              iot_hue_lightup      REFUSED (off-topic)    NO

  Saved all 80 lines to reports\f1_explained_20261024_1654_P1_A2-selected_0.csv (open it in Excel).
```

Steps 1 to 3: the two files, and the requests side by side (real output).

Why only 80 of the 100 requests? 20 are off-topic and have no correct intent; they are scored separately as off-topic F1. Look at request #15, "i need some more lights": the correct intent is iot_hue_lightup, but A2 **refused** it as off-topic. That is one of A2's 4 mistakes.

```
> python scripts/explain_f1.py   (steps 4 to 6)

STEP 4 and 5. For each intent, count and calculate

  TP = system said this intent and it was right
  FP = system said this intent but the correct one was different
  FN = this was the correct intent but the system said something else
  precision = TP/(TP+FP)   recall = TP/(TP+FN)   F1 = 2*P*R/(P+R)

  alarm_query
     TP 6 (#22, #34, #40, #41, #52, #64)   FP 0   FN 1 (#75)
     precision 6/6 = 1.000   recall 6/7 = 0.857   F1 = 0.923
  alarm_remove
     TP 7 (#35, #42, #44, #51, #53, #66, ...)   FP 0   FN 0
     precision 7/7 = 1.000   recall 7/7 = 1.000   F1 = 1.000
  ... (10 more intents) ...

STEP 6. Average the F1 of every intent (macro-F1)

  (0.923 + 1.000 + 1.000 + 1.000 + 1.000 + 0.923 + 1.000 + 1.000 + 1.000 + 0.923 + 1.000 + 0.923) / 12
  = 0.9744

  accuracy = 76 right / 80 = 0.9500

  Check against the other two ways of computing it:
    recorded in 20261024_1654_P1_A2-selected_0/metrics.json : 0.9744
    scikit-learn f1_score              : 0.9744
    -> all three are the same
========================================================================
```

Steps 4 to 6: counting, the formula for each intent, and the average (real output).

Read the alarm_query line like this: A2 got 6 alarm_query requests right (TP 6: requests #22, #34, #40, #41, #52, #64). It never said alarm_query wrongly (FP 0). It missed one, request #75 (FN 1). So precision = 6/6 = 1.000, recall = 6/7 = 0.857 and F1 = 2 × 1.000 × 0.857 / (1.000 + 0.857) = 0.923. Doing this for all 12 intents and averaging gives 0.9744. That matches the number recorded in metrics.json and the one scikit-learn computes.

Every run works the same way. For the generic prompt B1, for example:

```
python scripts/explain_f1.py --run 20261013_0110_P1_B1_0
```

Option 2: count it yourself in a spreadsheet

The script saved all 80 lines as `reports\f1_explained_20261024_1654_P1_A2-selected_0.csv`. Open it in Excel, or upload it to Google Sheets (sheets.google.com → File → Import). The columns are:

A	B	C	D	E	F
#	case_id	request	correct intent	system said	right?

Pick one intent, for example **alarm_query**, and count three things. You can use a filter (Data → Filter), or type these formulas into any empty cell:

Count	Meaning	Formula	Answer
TP	correct intent is alarm_query AND right	=COUNTIFS(D:D,"alarm_query",F:F,"yes")	6
FP	system said alarm_query AND wrong	=COUNTIFS(E:E,"alarm_query",F:F,"no")	0
FN	correct intent is alarm_query AND wrong	=COUNTIFS(D:D,"alarm_query",F:F,"no")	1

Then: precision = TP / (TP + FP) = 6/6 = 1.000; recall = TP / (TP + FN) = 6/7 = 0.857; F1 = 2 × precision × recall / (precision + recall) = 0.923. Repeat for the other 11 intents (only the name in the formula

changes), add the 12 F1 values and divide by 12. You get 0.974, the recorded score.

> **A refused request** ("REFUSED (off-topic)" in column E) counts as wrong for its correct intent (an FN), but as an FP for no intent, because the system did not claim any intent. The scorer and scikit-learn both treat it this way.

Option 3: a small practice example on paper

This table is made up for practice. It is not from the project. It has only 8 requests, so the whole calculation fits on paper. Once you can do this, you can do the real one above.

#	Correct intent	System said	Right?
1	alarm_set	alarm_set	yes
2	alarm_set	alarm_set	yes
3	alarm_set	alarm_remove	no
4	alarm_remove	alarm_remove	yes
5	alarm_remove	alarm_set	no
6	lights_off	lights_off	yes
7	lights_off	lights_off	yes
8	lights_off	alarm_set	no

Step 1. For each intent, count TP, FP and FN, exactly as in Option 2:

Intent	TP	FP	FN	Precision	Recall	F1
alarm_set	2 (#1, #2)	2 (#5, #8)	1 (#3)	2/4 = 0.500	2/3 = 0.667	0.571
alarm_remove	1 (#4)	1 (#3)	1 (#5)	1/2 = 0.500	1/2 = 0.500	0.500
lights_off	2 (#6, #7)	0	1 (#8)	2/2 = 1.000	2/3 = 0.667	0.800

Step 2. F1 for alarm_set = 2 × 0.500 × 0.667 / (0.500 + 0.667) = 0.667 / 1.167 = 0.571.

Step 3. Macro-F1 = (0.571 + 0.500 + 0.800) / 3 = **0.624**. Accuracy = 5 right / 8 = **0.625**.

Step 4. Check with scikit-learn: save this as `check_f1.py` in the project folder and run `python check_f1.py`. It prints 0.624 and 0.625.

```
from sklearn.metrics import f1_score, accuracy_score

gold = ['alarm_set', 'alarm_set', 'alarm_set', 'alarm_remove', 'alarm_remove',
        'lights_off', 'lights_off', 'lights_off']
pred = ['alarm_set', 'alarm_set', 'alarm_remove', 'alarm_remove', 'alarm_set',
        'lights_off', 'lights_off', 'alarm_set']

print('macro F1', round(f1_score(gold, pred, average='macro'), 3))
print('accuracy', accuracy_score(gold, pred))
```

Part B. Software tests: writing and running your own

B1. What an automated test is

A test is a short program with three parts: **set up** an input, **run** the code, **check** the result is what it must be. If the check fails, the test fails. Here is a real-style example:

```
def test_a_perfect_system_scores_1():
    # 1. set up: two requests and their correct answers
    cases = [
        {"case_id": "a", "utt": "set an alarm", "true_intent": "alarm_set",
         "in_scope": True, "entities": []},
        {"case_id": "b", "utt": "delete my alarm", "true_intent": "alarm_remove",
         "in_scope": True, "entities": []},
    ]
    answers = [
        Prediction("a", "ok", "alarm_set", [], True, True),
        Prediction("b", "ok", "alarm_remove", [], True, True),
    ]
    # 2. run the scorer
    result = score(cases, answers, labels=["alarm_set", "alarm_remove"])
    # 3. check: every answer is right, so F1 must be exactly 1.0
    assert result["primary"]["intent_macro_f1"] == 1.0
```

`assert` means "this must be true". The project's 371 tests are all built this way. The scorer's tests use small examples where the correct F1 was calculated by hand, like Part A4.

B2. Write and run your own first test (10 minutes)

1. In the project folder, open the `tests` folder and create a new file called `test_my_first.py`.
2. Paste this into it and save:

```
from sklearn.metrics import f1_score

from src.evaluation.scorer import Prediction, score


def test_a_perfect_system_scores_1():
    cases = [
        {"case_id": "a", "utt": "set an alarm", "true_intent": "alarm_set", "in_scope": True,
"entities": []},
        {"case_id": "b", "utt": "delete my alarm", "true_intent": "alarm_remove", "in_scope":
True, "entities": []},
    ]
    answers = [
        Prediction("a", "ok", "alarm_set", [], True, True),
        Prediction("b", "ok", "alarm_remove", [], True, True),
    ]
    result = score(cases, answers, labels=["alarm_set", "alarm_remove"])
    assert result["primary"]["intent_macro_f1"] == 1.0


def test_our_scorer_agrees_with_sklearn_on_one_mistake():
    cases = [
        {"case_id": "a", "utt": "set an alarm", "true_intent": "alarm_set", "in_scope": True,
"entities": []},
        {"case_id": "b", "utt": "delete my alarm", "true_intent": "alarm_remove", "in_scope":
True, "entities": []},
        {"case_id": "c", "utt": "wake me at six", "true_intent": "alarm_set", "in_scope":
True, "entities": []},
    ]
    answers = [
        Prediction("a", "ok", "alarm_set", [], True, True),
        Prediction("b", "ok", "alarm_remove", [], True, True),
        Prediction("c", "ok", "alarm_remove", [], True, True),  # the mistake
    ]
    ours = score(cases, answers, labels=["alarm_set", "alarm_remove"])["primary"]
["intent_macro_f1"]
    theirs = f1_score(["alarm_set", "alarm_remove", "alarm_set"],
                      ["alarm_set", "alarm_remove", "alarm_remove"], average="macro")
    assert abs(ours - theirs) < 1e-9
```

1. Run it. `-vv` lists each test by name:

```
python -m pytest tests/test_my_first.py -vv
```

```
> python -m pytest tests/test_my_first.py -vv

tests/test_my_first.py::test_a_perfect_system_scores_1 PASSED           [ 50%]
tests/test_my_first.py::test_our_scorer_agrees_with_sklearn_on_one_mistake PASSED [100%]
============================== 2 passed in 1.32s ===============================
```

Both tests pass (real output).

1. **Now break it on purpose:** change `== 1.0` to `== 0.9` and run it again. The first test now says **FAILED** and shows the real value (1.0). That is how tests catch mistakes.
2. Change it back to `1.0`. You can keep this file: it is your own test, and the total becomes 373.

The second test is a small version of the project's big independent check: our scorer and scikit-learn must agree.

B3. Run the project's own tests

```
python -m pytest tests                       # all 371, about 15 seconds
python -m pytest tests/test_scorer.py -vv    # the scorer's 14 tests, by name
```

```
> python -m pytest tests/test_scorer.py -vv   (first 8 of 14)

tests/test_scorer.py::test_fixture_gold_spans_are_self_consistent PASSED [  7%]
tests/test_scorer.py::test_perfect_system_scores_one_everywhere PASSED   [ 14%]
tests/test_scorer.py::test_macro_f1_is_one_when_averaged_over_only_the_supported_labels PASSED [ 21%]
tests/test_scorer.py::test_wrong_system_scores_zero_and_records_why PASSED [ 28%]
tests/test_scorer.py::test_empty_entity_output_does_not_look_successful PASSED [ 35%]
tests/test_scorer.py::test_malformed_and_failed_responses_stay_in_every_denominator PASSED [ 42%]
tests/test_scorer.py::test_a_failure_is_not_charged_as_a_false_positive_to_any_intent PASSED [ 50%]
tests/test_scorer.py::test_schema_validity_is_reported_before_and_after_repair PASSED [ 57%]
```

Some of the scorer's tests. Each name says what it checks.

Test file	What it checks
test_scorer.py	F1 and every score, against hand-calculated examples
test_massive.py	the dataset's answers were converted correctly
test_access.py	test data really is locked before the freeze
test_architect.py / test_architect_v2.py	the agent can only pick real designs and never exceeds 4 tries
test_runtime.py	the exported package refuses to load if tampered with
test_reproduce_metrics.py	the reproduction check catches a hand-edited number

Part C. Every experiment, one by one

Each experiment has the same seven parts: **in one sentence**, **where its files are**, **how it works**, **run it**, **what the output means**, **check it yourself** and **what to say**. All paths are inside `D:\MyClaudeProject\nlp`. Before running anything: open the Ollama app, open PowerShell, and type `cd D:\MyClaudeProject\nlp`.

> **Three tools do the "check it yourself" parts for you, on the real files:**
> `python scripts/explain_f1.py --run <run folder name>`: puts the correct answers and a run's answers side by side and works out its F1 (Part A5). It works for **any** run folder that has a predictions.jsonl.
> `python scripts/explain_stats.py bootstrap | timing | errors`: shows how the confidence interval, the speed figures and the error counts were worked out.
> A **run folder name** is a folder inside `runs`, such as `20261013_0110_P1_B1_0`: the date and time (13 Oct, 01:10), the project (P1), the system (B1) and the seed (0).

C1. The data and the splits

In one sentence: We used real, public data and split it once, with a fixed seed, into examples, development data and locked test data.

Where its files are:

What	Where
The original dataset (downloaded)	data\raw\massive-1.1.tar.gz (Amazon MASSIVE 1.1)
Which domains each project uses	configs\projects.json
The script that builds the splits	scripts\prepare_data.py
Code that turns the dataset's labels into positions	src\evaluation\massive.py
Examples, development and test files	data\processed\P1\kshot.jsonl, dev.jsonl, holdout\test.jsonl
The record of every split	data\manifests\P1.json (counts, every request ID, file fingerprints)
The data report	docs\DATA_REPORT.md
Tests for it	tests\test_massive.py, tests\test_data_splits.py

How it works:

1. Download MASSIVE: 16,521 English requests, each labelled by people with its intent and entities.
2. The dataset marks entities inside the text, like `wake me at [time : five am]`. Convert each one to its exact position: "five am" starts at character 11 and ends at 18. All 16,521 converted with 0 errors.
3. For each project, keep only its domains (P1: alarms and smart-home devices).
4. Draw, using a fixed seed so the result is identical every time: 10 examples per intent (kshot.jsonl), 100 development requests (80 in scope + 20 off-topic) and 300 locked test requests.
5. Save every request ID and a fingerprint (SHA-256 hash) of every file in the manifest, then check that no request appears in two files (the leakage check).

Run it (a few seconds; it only checks, it changes nothing):

```
python scripts/prepare_data.py --check
```

```
> python scripts/prepare_data.py --check

MASSIVE 1.1 en-US: 16521 records, licence CC BY 4.0 (Attribution 4.0 International)
P1: {'kshot': 120, 'dev_in_scope': 80, 'dev_oos': 20, 'dev_total': 100, 'test_in_scope': 250, 'test_oos': 50, 'test_total': 300} ...
P2: {'kshot': 100, 'dev_in_scope': 80, 'dev_oos': 20, 'dev_total': 100, 'test_in_scope': 250, 'test_oos': 50, 'test_total': 300} ...
P3: {'kshot': 60, 'dev_in_scope': 80, 'dev_oos': 20, 'dev_total': 100, 'test_in_scope': 167, 'test_oos': 33, 'test_total': 200} |...
```

The data check (real output).

What the output means:

- `kshot: 120`: 120 examples for P1's 12 intents (10 each).
- `dev_total: 100`, `test_total: 300`: the development and test sizes.
- `leakage id-check pass`: no request is in two files.
- P3's test set is smaller (200) because the original dataset has fewer requests in its domains; the plan's fallback size was used and recorded.

Check it yourself:

1. Open `data\processed\P1\dev.jsonl` in Notepad. The status bar shows 100 lines: one request each.
2. Find "make a room darker". Its entity is `"start": 7, "end": 11`. Count the characters from 0: m=0, a=1, k=2, e=3, space=4, a=5, space=6, r=7, o=8, o=9, m=10, so characters 7 to 10 spell "room" (the end number is one past the last letter).
3. Open `data\manifests\P1.json` and search (Ctrl+F) for `"counts"`: the sizes match the screen above. Search for `"sha256"` to see each file's fingerprint.

Say: "The data is the public MASSIVE dataset. Every split is drawn with a fixed seed and checked for overlap, and the test data is locked."

Where this comes from, and how it is compared

Who decided it: your project plan, section 4 (`docs\PROJECT_PLAN.md` lines 270 to 330). It proposed the three projects (P1 alarm and iot, P2 calendar, email and lists, P3 takeaway and transport), the split sizes, and the rule that off-topic requests come only from domains no project uses (cooking, news, weather, social...). The exact choices are written in `configs\projects.json`.

Why these settings: 10 examples per intent is the plan's "k = 10": every AI system gets the same small amount of data, the realistic case for a new app. 100 development and 300 test requests per project is what the plan set (lines 293 to 301). The official MASSIVE split labels are kept, so a test request in our files was also a test request in MASSIVE.

What it is compared with, and how: nothing: this experiment prepares the data every other experiment is scored on. Its checks are internal: every annotation converts exactly, every file's fingerprint is recorded, and no request is in two files.

The result: 16,521 / 16,521 annotations converted with 0 errors; the leakage check passes for all three projects. P3's test set is 200, not 300, because MASSIVE has only 181 in-scope test requests in its

domains; the plan's own fallback of 200 was used and recorded in `docs\DATA_REPORT.md`.

> **The test files are locked by code** (`src\analyzer\access.py`) until the freeze, and P3 is locked completely, so nothing so far has been tuned on test data.

C2. Model speed and repeatability

In one sentence: Before any experiment, we measured whether the AI model is fast enough and gives the same answer every time.

Where its files are:

What	Where
The script	scripts\throughput_test.py
The result folder	runs\20261012_0020_setup_throughput_0
Summary table	runs\20261012_0020_setup_throughput_0\summary.md
Time for every request	runs\20261012_0020_setup_throughput_0\timing.jsonl

How it works:

1. Switch off the saved answers, so every request really goes to the model.
2. Send 50 full-size requests and time each one with a stopwatch in the code.
3. Send the same 50 again and compare the two sets of answers word for word.
4. Count how many replies were valid JSON.

Run it (a few minutes: 100 live requests):

```
python scripts/throughput_test.py --models qwen2.5:3b
```

What the output means:

- median 0.42 s: half the requests took less than 0.42 seconds.
- identical repeat 1.00: all 50 answers were the same the second time.
- JSON valid 1.00: every reply was in the right format.

Check it yourself:

1. Open `runs\20261012_0020_setup_throughput_0\summary.md` in Notepad and read the table.
2. Open `timing.jsonl` in the same folder: one line per request with its time in seconds (`latency_s`).

Say: "The model answers in under half a second and gives identical answers on repeat, so anyone re-running our experiments gets the same results."

Where this comes from, and how it is compared

Who decided it: your project plan, section 2A, "Day-1 throughput test" (lines 187 to 195), and "Rules for the models" (line 177 onwards).

Why these settings: the plan asked for 50 realistic full-size requests, a second pass to check repeatability, and this rule for choosing the model: the largest model that is fast enough for the whole experiment and replies in valid JSON at least 95% of the time.

What it is compared with, and how: the plan wanted two models compared, a small one (about 3B) and a larger one (about 7 to 8B). Only qwen2.5:3b was measured: the 8B could not be downloaded on our connection, and it would not fit the laptop's 4 GB graphics card anyway. This is recorded as decision 39 and as a limitation.

The result: qwen2.5:3b passed both rules (JSON valid 1.00; the whole experiment fits in about 2.4 hours) and gave identical answers 50 of 50 times, so it became the model every AI-based system uses.

> **Why this matters for fairness:** the plan's rule "one runtime model for every LLM-based system" (line 179) means B1, B2, B4, A1 and A2 all use this same model, so differences between them come from the design, not the model.

C3. B0: word matching, no AI

In one sentence: The simplest possible system, as a floor that every AI system must beat.

Where its files are:

What	Where
The code	src\baselines\b0_tfidf.py
The script	scripts\run_b0.py
Result folders	runs\20261011_1614_P1_B0_0 (the first run) and later re-runs, e.g. runs\20261025_1518_P1_B0_0

How it works:

1. Turn every example sentence into word weights (TF-IDF: a word counts more if it is common in this sentence but rare overall).
2. Train a standard classifier (logistic regression, from scikit-learn) on P1's 120 examples.
3. For off-topic requests: if a request is too unlike every example, refuse it. The cut-off (0.3482) was chosen on development data.
4. Answer the 100 development requests and score them.

Run it (a few seconds; no AI model needed):

```
python scripts/run_b0.py --projects P1
```

What the output means:

- `intent macro-F1 0.780 | accuracy 0.700`: the scores.
- `threshold 0.3482`: the cut-off for refusing off-topic requests.
- `entity F1 0.000 (none extracted)`: B0 only picks an intent; it never extracts entities.

Check it yourself:

1. Work out its F1 from the real files: `python scripts/explain_f1.py --run 20261011_1614_P1_B0_0`. It ends with 0.7801 (0.780 when rounded) and "all three are the same".
2. Every re-run makes a new dated folder in `runs` with exactly the same score: that shows the result repeats.

Say: "A basic word-matching classifier gets 0.78 on P1; any AI system has to beat that to be worth it."

Where this comes from, and how it is compared

Who decided it: your project plan, line 250: "B0 = Simple reference: TF-IDF intent classifier with a documented OOS threshold … sanity reference, not automatically SOTA".

Why these settings: it is the simplest reasonable system that needs no AI model, so it shows how much the AI adds. Its off-topic cut-off (0.3482) was chosen on development data, which the plan allows; that also makes its development score slightly optimistic.

What it is compared with, and how: every other system, on the same 100 development requests with the same scorer. It is a floor: a system that cannot beat B0 is not worth using.

The result: 0.780 on P1 and 0.730 on P2. Every AI-based system beats it, and in the robustness test (C13) it falls much further than the AI systems when requests are messy.

> B0 never extracts entities, so its entity F1 is 0 by design, not by failure.

C4. B1: one generic prompt

In one sentence: What you get from one standard AI prompt with no design decisions.

Where its files are:

What	Where
The design it uses	joint-fixed-llm, fixed in advance (B1_CONFIG_ID in src\pipelines\catalogue.py)
The script	scripts\run_design.py
Result folder	runs\20261013_0110_P1_B1_0 (predictions.jsonl, metrics.json)

How it works:

1. Use one design, chosen before any results were seen.
2. Run it on the 100 development requests, following the recipe in Part A2.
3. Save every answer and the scores in a new run folder.

Run it (under a minute):

```
python scripts/run_design.py --systems B1 --projects P1
```

What the output means:

- `B1 joint-fixed-llm intent 0.927 entity 0.610 oos 0.870`: its intent, entity and off-topic F1.

Check it yourself:

1. `python scripts/explain_f1.py --run 20261013_0110_P1_B1_0` shows its 80 answers, 8 of them wrong, and works out 0.9268.
2. In the CSV file it saves, filter column F for "no" to see exactly which 8 requests it got wrong.

Say: "One generic prompt reaches 0.927; the agent has to beat this to show its design work adds anything."

Where B1 comes from, and how it is compared with our agent

Who decided it: your project plan. B1 was defined in the project plan, before any code was written, in `docs\PROJECT_PLAN.md`:

- **Line 251:** "B1 = Generic LLM pipeline with a fixed prompt and procedure … receives supported labels/schema … no project-specific search."
- **Line 411:** "B1 is fixed to one predeclared configuration (joint single call, fixed examples, LLM label)."

"Predeclared" means it was chosen **in advance, before any results were seen**, so it could not be picked to make the agent look good.

Where it is in the code. That line of the plan became one line in `src\pipelines\catalogue.py` (line 39):

```
B1_CONFIG_ID = "joint-fixed-llm"
```

It simply names one of the 12 designs: **joint** (one AI call), **fixed** (the same one example per intent every time) and **llm** (the model itself may answer none_of_the_above). It is the plainest combination of the three choices. The prompt wording comes from the `render_prompt` function in the same file; **all 12 designs share that template**, and B1 is its simplest setting.

What "generic" means here. Not a famous prompt from somewhere else. It means one fixed design, used the same way on every project (P1, P2, or any new one), with no project-specific decisions.

	B1 gets it?	Our agent gets it?
The project's list of intents and fields	yes	yes
10 labelled examples per intent	yes (shows 1 per intent)	yes
The same AI model and the same 100 test requests	yes	yes
Descriptions written by the Analyzer	no	yes
Choosing between the 12 designs	no: always joint-fixed-llm	yes: ranks, tests, revises
Testing designs and keeping the best	no	yes

An example: a hospital project. Say the app is a hospital assistant with intents such as book_appointment, cancel_appointment and check_bed_availability. B1 would always use the same template, filled in with the hospital's intents and one example each:

```
Label this request for a hospital assistant. Reply with JSON only.

Allowed intents:
- book_appointment
- cancel_appointment
- check_bed_availability
- none_of_the_above  (use this if the request is about anything else)

Examples:
Request: I want to see a cardiologist on Monday
Answer: {"intent": "book_appointment"}
...

Request: <the patient's request>
Answer:
```

Our agent would read the hospital's description, write a description of every intent and field, and might decide that patients phrase things in many ways, so the most similar examples and written descriptions help (schema-sim-llm). Then it tests that choice before keeping it. Both would then be scored on the same labelled hospital requests. For a new project you need the same ingredients we took from MASSIVE: a short description, the intents and fields, about 10 labelled examples per intent, and labelled test requests kept aside.

How the comparison actually runs. The script `scripts\run_design.py` does both, one after the other, for each project:

Step	B1 (from line 137 of run_design.py)	Our agent (A1 / A2)
Design	takes joint-fixed-llm, always	analyses the project and chooses a design
Requests	the same 100, from data\processed\P1\dev.jsonl	the same 100
AI model	qwen2.5:3b	qwen2.5:3b
Scoring	the same score() function	the same score() function
Saved in	runs\20261013_0110_P1_B1_0	runs\20261024_1654_P1_A1-selected_0 and ...A2-selected_0

The rule, also fixed in advance (`docs\PROJECT_PLAN.md` line 389): the agent counts as better than B1 only if (1) it gains **at least +2.0 points** of intent F1, (2) it loses **no more than 2.0 points** of entity F1, (3) at least 98% of its replies are valid, and (4) it is no more than twice as slow as B1.

System	Project	Intent change (needs +2.0 or more)	Entity change (needs -2.0 or better)	Passes?
A1	P1	+3.2	-1.3	yes
A1	P2	+3.8	**-8.6**	**no**: its entity extraction drops too much
A2	P1	+4.8	+9.7	yes
A2	P2	+9.4	+3.3	yes

> **Be accurate about this.** The original agent (A1) beats B1 on **intent** on both projects, but under the plan's full rule it **fails on P2**, because its entity F1 is 8.6 points below B1's. The improved agent (A2) passes on both projects. All of these are development numbers; the rule is applied again to the locked test data after the freeze. This is recorded as finding F9 in docs\FINDINGS.md.

See both sides yourself, request by request:

```
python scripts/explain_f1.py --run 20261013_0110_P1_B1_0
python scripts/explain_f1.py --run 20261024_1654_P1_A1-selected_0
```

The same 80 requests appear in both, each with the system's answer next to the correct one. To see B1's real prompt, run `python scripts/walkthrough.py`: its default design is B1's, and step 3 shows the prompt.

Say: "B1 is the default: one standard prompt, chosen in our plan before any results, used the same way for every project. Our agent and B1 answer the same requests with the same model and scorer, and our plan fixed in advance how much better the agent must be. By that rule the improved agent passes on both projects; the original failed on P2 because of entity extraction."

C5. B2: a person configuring by hand

In one sentence: A team member configured the pipeline by hand, timed, so we can compare the agent with a person.

Where its files are:

What	Where
The worksheets	configs\b2\P1_worksheet.md, configs\b2\P2_worksheet.md
The script	scripts\run_b2.py (start, check, finish)
Result folders	runs\20261024_1622_P1_B2_0, runs\20261024_1630_P2_B2_0
The time and help declared	inside each result folder's config.json ("effort", "ai_assistance")

How it works:

1. `run_b2.py start` created a worksheet with the project description and the 12 designs.
2. The person chose a design, wrote a description for every intent and field, and declared any help.
3. `run_b2.py check` scored the choices on development data; each check is added to the worksheet.
4. `run_b2.py finish` saved the final result with the minutes and the help recorded.

Run it (the person's own time: 38 min for P1, under 6 min for P2):

```
python scripts/run_b2.py check --project P1
```

What the output means:

- `intent macro-F1 0.858 | entity F1 0.586`: the person's scores on P1.

Check it yourself:

1. Open `configs\b2\P1_worksheet.md`: section 2 shows the chosen design, sections 3 and 4 the descriptions, section 5 the time and help, and the bottom shows each check's score.
2. Open `runs\20261024_1622_P1_B2_0\config.json` and search for `"ai_assistance"`.
3. `python scripts/explain_f1.py --run 20261024_1622_P1_B2_0` works out 0.858 from its answers.

Say: "By hand, a beginner got 0.858 on P1 in 38 minutes; the agent got 0.958 with no human time. On P2, a person with ChatGPT matched the agent, and we report that too."

Where this comes from, and how it is compared

Who decided it: your project plan, line 252: "B2 = Manually configured project-specific pipeline … human picks a configuration and hand-writes intent/entity descriptions; 2-hour budget per project, recorded". The plan made B2 part of the minimum result (line 453) because it answers research question RQ2 (line 238): how does the agent compare with a manually designed pipeline, considering quality and effort?

Why these settings: the person uses the same 12 designs, the same AI model and the same scorer as the agent, so only the decisions differ. The 2-hour budget stops endless trying. Any help must be declared, because the same team built the agent.

What it is compared with, and how: the agent (A1 and A2), on the same 100 development requests, on two things: the score and the human time it took.

Project	Person (B2)	Time	A1	A2
P1	0.858 (alone)	38 min	0.958	0.974
P2	0.861 (with ChatGPT)	under 6 min	0.841	0.898

The result: on P1, a person working by hand took 38 minutes and scored 0.858; the agent scored 0.958 (A1) and 0.974 (A2) with no human time. On P2, the person used ChatGPT (declared) and scored 0.861 in under 6 minutes, slightly above A1 (0.841) and below A2 (0.898).

> **Read these carefully.** Each project was measured once, by a beginner on the team that built the agent, so it is not a blind test. P1 and P2 measure different things: a person alone, and a person helped by ChatGPT. On P2 the person estimated 10 to 20 minutes, but the worksheet's own timestamps show under 6 minutes, so the measured time is the one reported (finding F7).

C6. B3: the published baseline (XLM-R)

In one sentence: We compared against the model the MASSIVE paper itself used as its baseline.

Where its files are:

What	Where
The code	src\baselines\b3_encoder.py
The script	scripts\run_b3_local.py
The downloaded model	C:\Users\mihir\.cache\huggingface\hub\models--xlm-roberta-base (1.1 GB)
Result folders	runs\20261015_2240_P1_B3-k_0 and the B3-full and P2 folders
Summary of each run	summary.md inside each folder

How it works:

1. Download XLM-R, a language model with 270 million parameters.
2. Fine-tune it: show it the examples again and again (30 rounds) and adjust its internal numbers until it learns them. B3-k uses the same 10 examples per intent as everyone else; B3-full uses all the training data (10-30 times more).
3. Choose the off-topic cut-off on development data.
4. Answer the 100 development requests and score them with the same scorer as every other system.

Run it (about 11 minutes for P1 B3-k; about 85 minutes for all four. Don't run it live: open the result folder instead):

```
python scripts/run_b3_local.py --projects P1 --variants B3-k
```

What the output means:

- The summary says: fine-tuned on 120 cases for 30 epochs in 664 s, intent macro-F1 0.909, entity F1 0.650.

Check it yourself:

1. Open `runs\20261015_2240_P1_B3-k_0\summary.md` in Notepad.
2. `python scripts/explain_f1.py --run 20261015_2240_P1_B3-k_0` works out 0.909 from its saved answers.

Say: "With the same data, our agent beats the published baseline on intents, and A2 also beats it on entities. B3-full, with 10-30 times more data, is only a reference."

Where this comes from, and how it is compared

Who decided it: your project plan, line 253 and line 258: B3 follows the MASSIVE paper's own published reference baseline (XLM-R), so the project has a comparison with published work, which DA2 requires.

Why these settings: the plan asked for two versions (line 264, "data regime"): **B3-k**, trained on the same 10 examples per intent as every other system, which is the fair comparison; and **B3-full**, trained on all the training data (10 to 30 times more), shown only as an upper reference. Like B0, it refuses off-topic requests with a confidence cut-off chosen on development data.

What it is compared with, and how: the AI systems, on the same 100 development requests with the same scorer. Only B3-k is a matched comparison.

The result: with the same data (B3-k), the agent wins on intent (P1: A1 0.958 and A2 0.974 against 0.909; P2: 0.841 and 0.898 against 0.833) and on refusing off-topic requests. On entities, B3-k beats A1 but A2 beats B3-k (P1 0.707 against 0.650; P2 0.483 against 0.448). B3-full wins on entities, as expected with far more data.

> **Do not call B3-full a fair comparison**: it learns from 10 to 30 times more data. The plan says to report it plainly as an upper reference (line 264).

XLM-R explained simply: what it is, and how it compares with our system

What XLM-R is. A **language model**: a program that has read huge amounts of text in about 100 languages and learned how words relate to each other. It is **not a chatbot**: it cannot chat or write answers. The creators of the MASSIVE dataset used it as their own reference system in their published paper.

What it knows, and what it does not. It knows English. It does **not** know our intents: it has never heard of iot_hue_lightoff or book_appointment. And it does **not** build pipelines; it cannot build anything.

What we do with it, step by step:

1. Load XLM-R. It knows language, but nothing about our app.
2. Add a small extra layer on top, with one output per intent (12 for P1).
3. **Train it** on our examples: show it the 120 labelled sentences 30 times. Each time it gets one wrong, its internal numbers are adjusted a little. This is **fine-tuning**; it took about 11 minutes.
4. After training, **it is the pipeline**: give it a sentence and it directly answers with the intent. No prompt, no instructions.

Two ways to build the part that understands requests



Way 1 changes the model. Way 2 changes only the text sent to it; our framework's job is to find the best text.

Way 1 (XLM-R) changes the model by training it. Way 2 (our framework) changes only the text sent to qwen.

An everyday picture. Think of a new waiter who speaks English but does not know your menu. **XLM-R is a waiter who memorises the menu** before starting work: their brain changes. **Our system is a waiter who is handed a note** every time a customer speaks ("these are the only items on our menu... example:

'something warm with milk' means coffee"): their brain never changes, and our framework's job is to write the best note.

"It has learned the intents, so what?" Now it can sit **the same exam** as our system. Both have exactly the same job, reading a request and saying its intent, so both answer the same 80 development requests and are scored the same way. Real answers on P1:

Request	Correct	XLM-R (B3-k) said	Our system (A2) said
turn the lights off please	iot_hue_lightoff	right	right
make a room darker	iot_hue_lightdim	wrong: iot_cleaning	right
switch on the lights	iot_hue_lighton	wrong: refused it	right
start up the coffee machine	iot_coffee	wrong: refused it	right
please remove any alarms that i have set	alarm_remove	wrong: refused it	right
start new smart socket	iot_wemo_on	right	wrong: refused it

The full count on 80 requests: both right on 68; only our system right on 8; only XLM-R right on 1. As F1: our system (A2) 0.974, XLM-R 0.909.

Why XLM-R struggled: 10 examples per intent is very little to learn from, so it was often unsure and refused normal requests. qwen did not need to learn the intents, because they are written on its note.

	XLM-R (B3)	Our framework (with qwen)
Already knows language?	yes	yes
Knows our intents at the start?	no	no, but they are listed in the prompt
How it gets our intents	**trained** on our examples: its insides change	**told** them in the prompt: qwen never changes
Who builds the pipeline	a person runs the training	**our program, automatically**
What the pipeline is	the trained XLM-R model	a recipe (the chosen prompt design) plus qwen
A new project	train again from scratch	the framework designs a new prompt automatically

So what the comparison tells us: for a new app with only about 10 examples per intent, giving a ready-made AI the right prompt (our way) got more answers right than training the published model (the standard way), and needed no training. The full plain-language explanation is in the separate document "How Our System Works".

C7. B4: random search

In one sentence: We checked whether the agent chooses designs better than luck.

Where its files are:

What	Where
The code	src\baselines\b4_search.py
The script	scripts\run_design.py
Random-search results	runs\20261013_0112_P1_B4-random_seeds (config.json, metrics.json)
All 12 designs scored	runs\20261013_0112_P1_B4-exhaustive_0 (config.json)

How it works:

1. Pick 4 of the 12 designs at random: the same budget the agent gets.
2. Score each on development data and keep the best, by the same rule the agent uses.
3. Repeat with 5 different random seeds, so the result is not one lucky or unlucky draw.
4. Also score all 12 designs: the best possible (the ceiling).

Run it (under a minute):

```
python scripts/run_design.py --systems B4-random --exhaustive --projects P1
```

What the output means:

- For each seed, the 4 designs drawn and the one kept; then the average over the 5 seeds.

Check it yourself:

1. Open `runs\20261013_0112_P1_B4-random_seeds\metrics.json` and find `"selected_intent_macro_f1"`: the 5 winning scores, to 4 decimals, are 0.9585, 0.9511, 0.9744, 0.9511 and 0.9585.
2. Average them on a calculator: (0.9585 + 0.9511 + 0.9744 + 0.9511 + 0.9585) / 5 = 4.7936 / 5 = 0.9587, which rounds to **0.959**, the B4 score. (Use 4 decimals: rounding first to 3 gives 0.958.)
3. Open `config.json` in the same folder and search for `"attempted"` to see which 4 designs each seed drew.
4. Open the B4-exhaustive folder's `config.json` and search for `"ranking"`: all 12 designs, best first (schema-sim-llm, 0.974).

Say: "Random search averaged 0.959 on P1. A1 did not beat it; the improved A2 reached the best design, 0.974."

Where this comes from, and how it is compared

Who decided it: your project plan, line 254 ("B4-random: 4 configurations drawn at random (5 seeds). B4-exhaustive: all 12") and line 262, "Why B4 matters". It answers research question RQ5 (line 244): does the agent choose designs better than plain search with the same budget?

Why these settings: 4 designs because that is exactly the agent's budget (3 first choices plus 1 revision), so the comparison is equal. 5 seeds so the result is not one lucky or unlucky draw. Trying all 12 shows the best possible score.

What it is compared with, and how: the agent. Both use the same 12 designs, the same Analyzer descriptions, the same development requests, the same scorer and the same selection rule (`src\evaluation\selection.py`), so the only difference is how the 4 designs are chosen. The plan

explains why this matters: if the agent beats B1, that might only be because trying designs on development data helps; B4 tests whether the agent's own choices add anything.

The result: on P1 random search averaged 0.959 and on P2 0.878. A1 (0.958, 0.841) did not beat it, which is reported as finding F1. A2 (0.974, 0.898) reached the best of all 12 designs on both projects, above random search's average.

> **A2 was designed after seeing these results** (decision 42), so its win here is not proof on its own; only the locked test data after the freeze can confirm it.

C8. A1 and A2: the agent

In one sentence: We ran the full framework: the Analyzer, the Architect, scoring the candidates and selecting the best.

Where its files are:

What	Where
The Analyzer	src\analyzer\analyzer.py
A1's Architect	src\architect\architect.py
A2's Architect	src\architect\architect_v2.py
Puts the steps together	src\architect\design_run.py
Scripts	scripts\run_design.py (A1), scripts\run_a2.py (A2)
Result folders	runs\20261013_0110_P1_A1_0, runs\20261024_1648_P1_A2_0 (config.json, events.jsonl)
Answers of the chosen design	runs\20261024_1654_P1_A1-selected_0, runs\20261024_1654_P1_A2-selected_0

How it works:

1. Analyzer: ask the AI to write a one-line description of every intent and field (2 calls).
2. Architect: ask the AI which designs to try. A1 asks it to pick 3 at once; A2 asks it to rank each design choice separately, then builds 3 designs that each change one choice.
3. Score each of the 3 designs on the 100 development requests.
4. Give the scores back to the Architect, which picks a 4th design.
5. Keep the best of the 4. Save every choice, its reason and its score in the run folder.

Run it (under a minute):

```
python scripts/run_a2.py
```

```
> python scripts/run_a2.py

A2: runtime qwen2.5:3b (357c53fb659c), planner qwen2.5:3b (357c53fb659c)
=== P1: 20261025_1514_P1_A2_0 ===
  ranked extraction         schema_guided > two_stage > joint  (Schema-guided extraction provides clear instructions for the mode...
  ranked example_selection  fixed > similarity  (Fixed examples ensure consistency and coverage, while similarity selec)
  ranked oos_policy         tfidf_threshold > llm_label  (Tfidf threshold policy is cheaper and can prevent unnecessary model ca)
  tried schema-fixed-tfidf     intent 0.845 entity 0.505 [reasoned/initial]
  tried schema-sim-tfidf       intent 0.869 entity 0.617 [reasoned/initial]
  tried twostage-fixed-tfidf   intent 0.839 entity 0.544 [reasoned/initial]
  tried schema-sim-llm         intent 0.974 entity 0.707 [reasoned/revision]
  selected schema-sim-llm: intent 0.974 vs A1 0.958 -> meets the keep rule | 4/4 reasoned, 2 planning calls
=== P2: 20261025_1514_P2_A2_0 ===
  ranked extraction         schema_guided > two_stage > joint  (Schema-guided extraction provides clear instructions for each int...
  ranked example_selection  fixed > similarity  (Fixed examples cover all intents but ignore the nuances of the request)
  ranked oos_policy         tfidf_threshold > llm_label  (Tfidf threshold policy rejects requests early, saving resources and av)
  tried schema-fixed-tfidf     intent 0.762 entity 0.391 [reasoned/initial]
  tried schema-sim-tfidf       intent 0.802 entity 0.437 [reasoned/initial]
  tried twostage-fixed-tfidf   intent 0.772 entity 0.413 [reasoned/initial]
  tried schema-sim-llm         intent 0.898 entity 0.483 [reasoned/revision]
  selected schema-sim-llm: intent 0.898 vs A1 0.841 -> meets the keep rule | 4/4 reasoned, 2 planning calls
Decision 42 keep-or-drop rule: A2 GOES to the test set (P1: meets, P2: meets)
```

The A2 run (real output).

What the output means:

- `ranked extraction schema_guided > two_stage > joint`: the AI's own ranking, with its reason in brackets.
- `tried ... [reasoned/initial]`: the 3 first designs and their scores; `[reasoned/revision]`: the 4th, chosen after seeing the scores.
- `selected schema-sim-llm: intent 0.974`: the design kept.

Check it yourself:

1. Open `runs\20261013_0110_P1_A1_0\config.json` and search for `"attempts"`: the 4 designs A1 tried, each with a `"rationale"` (its reason) and `"source": "reasoned"` (the AI chose it, not a fallback).
2. Search the same file for `"reasoned_choices"`: 4, so all 4 choices came from the AI.
3. `python scripts/explain_f1.py --run 20261024_1654_P1_A2-selected_0` works out A2's 0.974 (Part A5).

Say: "The agent chooses designs itself and records a reason for each choice. A2 tests the alternatives one change at a time, and it reached the best of all 12 designs on both projects."

Where this comes from, and how it is compared

Who decided it: A1 is defined in your project plan (line 256, and section 6, line 413 onwards: the Architect with a budget of 3 + 1 designs). A2 was added later, after the development results showed the Architect's choices were the weak point: decision 42 (build A2 alongside A1) and decision 43 (A2 uses the same 3B model for planning) in `docs\decisions.md`.

Why these settings: A2 changes only how the Architect asks the AI: it ranks each design choice separately, tests the alternatives one change at a time, and changes one choice in its revision. Everything else (designs, model, data, scorer, selection rule) is identical to A1, so any difference comes from the search method.

What it is compared with, and how: B1 (the plan's full rule, see C4), B4 random search (C7), B2 the person (C5) and B3 the published baseline (C6), all on the same development requests with the same

scorer. A rule fixed **before** building A2 decides whether A2 goes to the final test: its development score must be at least A1's on both projects. It passed.

The result: A1: 0.958 and 0.841. A2: 0.974 and 0.898, the best of all 12 designs on both projects.

> **Two things to state honestly.** (1) A2's development scores are not proof, because A2 was designed after seeing them; A1 and A2 will both be reported on the locked test data, whichever does better. (2) The plan said the Architect could use a free online model and that each design run should be repeated three times (line 391). We used the local model only, and the three repeats were not run separately. That is a deviation from the plan, recorded as decision 44: the re-runs on 24 and 25 October reused the saved answers, so they do not show the model repeating itself. The evidence for that is the C2 test (50 of 50 identical short answers); long written reasons can still vary slightly in wording, as the mini agent in G8 shows.

C9. Ablations

In one sentence: We removed one part of the agent at a time to find out which part matters.

Where its files are:

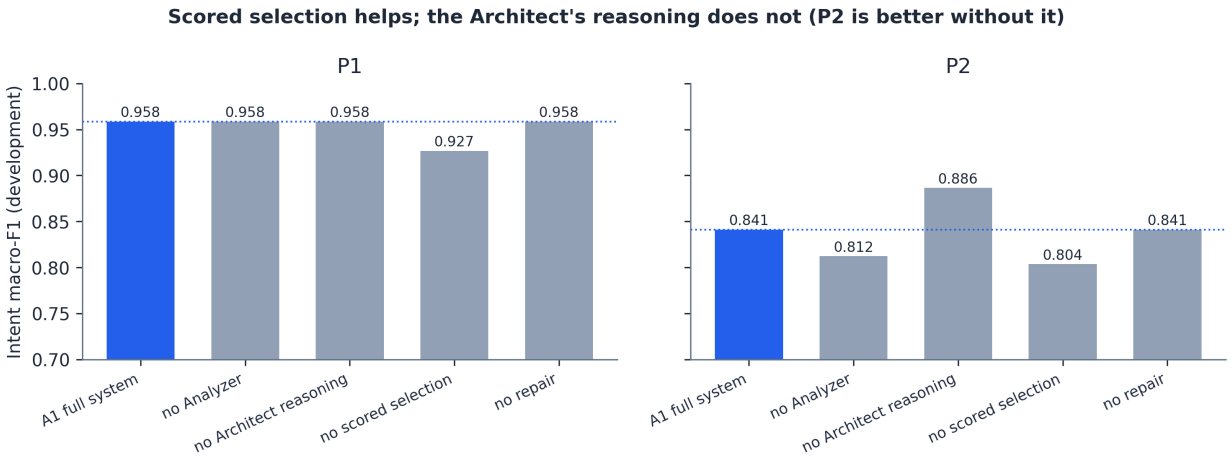
What	Where
The script	scripts\run_dev_analysis.py
One result folder per variant	runs\20261016_0100_P1_A1-noAnalyzer_0, ...-noReasoning_0, ...-noSelection_0, ...-oneCandidate_0, ...-noRepair_0
All numbers together	reports\dev_analysis.json (section "ablations")
The write-up	reports\DEV_ANALYSIS.md, section 1

How it works:

1. No Analyzer: run the agent without the AI-written descriptions.
2. No reasoning: replace the AI's choice of designs with a fixed order.
3. No selection: take the Architect's first design without scoring any.
4. No repair: give no second chance when the model's reply is badly formatted.
5. Score each variant on the same 100 requests and compare it with the full agent.

Run it (1 to 2 minutes):

```
python scripts/run_dev_analysis.py --skip-timing
```



Each bar removes one part of A1 (chart from the reports\figures folder).

What the output means:

- One line per variant, e.g. `A1-noReasoning joint-sim-llm intent 0.886` on P2: without the AI's reasoning, P2 scored higher than with it.

Check it yourself:

1. Pick a variant and work out its score: `python scripts/explain_f1.py --run 20261016_0102_P2_A1-noReasoning_0` gives 0.886.
2. Compare with the full agent on P2: `python scripts/explain_f1.py --run 20261024_1654_P2_A1-selected_0` gives 0.841.

Say: "Scoring the candidates helps most, by 3 to 4 points. A1's reasoning did not help, which is exactly why we built A2."

Where this comes from, and how it is compared

Who decided it: your project plan, section 8 (line 457 onwards) lists the ablations: no Analyzer, one fixed candidate, no evaluation-driven selection, and no repair. These answer research question RQ3 (line 240): what does each part contribute? One more, **no reasoning** (a fixed order instead of the AI's choice), is not in the plan; we added it because it isolates the Architect's own judgement.

Why these settings: each variant removes exactly one part and keeps everything else, so a change in score can only come from that part.

What it is compared with, and how: the full agent (A1), on the same 100 development requests with the same scorer.

The result: scored selection helps most (+3.2 on P1, +3.8 on P2). The Analyzer helps on P2 (+2.9). Repair hardly matters for this model. Removing the AI's reasoning did not hurt, and on P2 it helped (0.886 against 0.841), which led to A2.

> **These are development-stage ablations.** The plan runs them again on the locked test data after the freeze, and those are the ones the paper reports.

C10. Confidence intervals (bootstrap)

In one sentence: We checked whether a difference between two systems is real or could be luck.

Where its files are:

What	Where
The code	paired_bootstrap in scripts\run_dev_analysis.py
The results	reports\dev_analysis.json (section "bootstrap")
The step-by-step explainer	scripts\explain_stats.py

How it works:

1. Take the 100 development requests and both systems' answers to them.
2. Make a "pretend" set of 100 by drawing request numbers at random, allowing repeats: some requests appear twice, some not at all.
3. Score both systems on that same pretend set and note the difference.
4. Do this 1,000 times. Sort the 1,000 differences and keep the middle 95%: that range is the confidence interval.
5. If the range is entirely above 0, the lead is real. If it includes 0, it could be luck.

Run it (about 20 seconds):

```
python scripts/explain_stats.py bootstrap
```

```
> python scripts/explain_stats.py bootstrap

======================================================================
Bootstrap: is A1 really better than B1 on P1, or could it be luck?
  A1 answers: runs\20261024_1654_P1_A1-selected_0\predictions.jsonl
  B1 answers: runs\20261013_0110_P1_B1_0\predictions.jsonl
======================================================================

STEP 1. On the real 100 requests: A1 0.958 - B1 0.927 = difference +0.032

STEP 2. Make a 'pretend' set of 100 requests by drawing request numbers at random,
        allowing repeats. Score BOTH systems on that same pretend set. Three examples:

  draw 1: requests #42, #20, #51, #84, #7, #10, #69, #13, #47, #75, ... (38 repeats, 38 never drawn)
          A1 0.886  B1 0.951  difference -0.065
  draw 2: requests #10, #98, #72, #74, #41, #44, #89, #45, #77, #64, ... (32 repeats, 32 never drawn)
          A1 0.974  B1 0.960  difference +0.014
  draw 3: requests #52, #51, #14, #62, #82, #52, #8, #25, #9, #27, ... (39 repeats, 39 never drawn)
          A1 0.957  B1 0.910  difference +0.048

STEP 3. Do this 1000 times (fixed seed 20261016), sort the 1,000 differences,
        and keep the middle 95%: that range is the 95% confidence interval.

  interval = [-0.040, +0.121]   A1 better in 85% of the 1,000 draws
  recorded in reports/dev_analysis.json: [-0.040, +0.121]  -> same

STEP 4. Read it: the interval includes 0, so with only 100 requests A1's lead over B1
        could still be luck. That is why the paper does not call it significant yet.
======================================================================
```

The bootstrap, step by step (real output).

What the output means:

- Step 1: on the real 100 requests A1 leads B1 by +0.032.
- Step 2: three of the 1,000 pretend sets. In draw 1 B1 actually wins (-0.065): that is how luck shows up.
- Step 3: the interval is [-0.040, +0.121]: it includes 0.

- It matches the number recorded in reports\dev_analysis.json.

Check it yourself:

1. Run it for P2: `python scripts/explain_stats.py bootstrap --project P2`.
2. Open `reports\dev_analysis.json` and search for `"ci_low"` to see every recorded interval.

Say: "With 100 development requests, A1's lead over B1 could still be luck, so we don't call it significant yet. The 300-request test sets will tell."

Where this comes from, and how it is compared

Who decided it: your project plan, line 391: "Use paired bootstrap confidence intervals for score differences on the same test examples and report per-project results."

Why these settings: paired means both systems are scored on the same pretend set each time, because they answered the same requests; that is fairer than resampling each separately. 1,000 draws is the usual number. The fixed seed makes the interval come out the same every time.

What it is compared with, and how: every main pair: A1 against B1, A1 against B3-k, and A1 against the best of the 12 designs, separately for P1 and P2.

The result: every interval includes 0 on development data: with only 100 requests per project, none of the differences is certain yet. The 300-request test sets will narrow them.

> **With only three projects, the plan says to make no claim about all projects in general** (line 391): the intervals describe these projects only.

C11. Real speed (timing)

In one sentence: We measured how fast the final pipeline answers when nothing is saved in advance.

Where its files are:

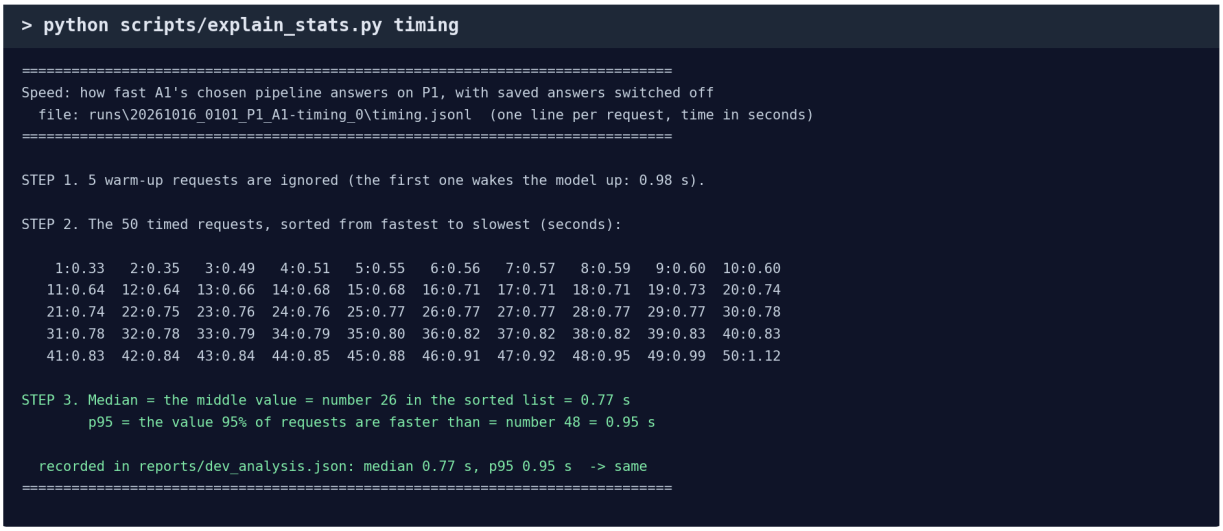
What	Where
The code	time_requests in src\evaluation\runner.py
Time for every request	runs\20261016_0101_P1_A1-timing_0\timing.jsonl
The step-by-step explainer	scripts\explain_stats.py

How it works:

1. Switch off the saved answers so every request goes to the model live.
2. Send 5 warm-up requests and ignore them: the first one wakes the model up and is slow.
3. Send 50 development requests and time each one with a stopwatch in the code.
4. Leave out requests refused before reaching the model, so they cannot make a design look faster.
5. Sort the 50 times. The median is the middle one; p95 is the one that 95% are faster than.

Run it (a second (it reads the saved times)):

```
python scripts/explain_stats.py timing
```



How the speed figures are worked out (real output).

What the output means:

- The 50 times sorted, numbered 1 to 50.
- Median = number 26 = 0.77 s. p95 = number 48 = 0.95 s.
- Both match what is recorded in reports\dev_analysis.json.

Check it yourself:

1. Open `runs\20261016_0101_P1_A1-timing_0\timing.jsonl` in Notepad: each line has a request and its `latency_s` (seconds). The first 5 say `"warmup": true`.
2. To measure again from scratch (about 5 minutes): `python scripts/run_dev_analysis.py`.

Say: "Every pipeline answers in under one second, even in the slowest 5% of requests."

Where this comes from, and how it is compared

Who decided it: your project plan, line 210: "Latency comes from separate timing runs: for each system and project, 50 fixed development cases run with the cache disabled, after a warm-up of 5 requests." The success rule (C4) also needs it: the agent may be at most twice as slow as B1 at p95.

Why these settings: speed must come from live requests, never from saved answers, which would look instant. The warm-up is dropped because the first request wakes the model. Requests refused before reaching the model are left out, so they cannot make a design look faster.

What it is compared with, and how: B1, on the same 50 requests: the agent's p95 must be no more than 2 x B1's.

The result: P1: A1 0.77 s median, 0.95 s p95 against B1's 0.44 s and 0.59 s (A1 uses 2 calls per request). P2: A1 0.43 s and 0.77 s against B1's 0.41 s and 0.79 s. A1 is within the 2 x limit on both projects.

> **A2 has not been timed yet.** Its chosen design (schema-sim-llm) makes one call per request, so it should be close to B1, but that is measured at the freeze, not assumed.

C12. Error analysis

In one sentence: We looked at what kind of mistakes the agent makes, not just how many.

Where its files are:

What	Where
The code	classify_errors in scripts\run_dev_analysis.py
The results	reports\dev_analysis.json (section "errors")
The explainer	scripts\explain_stats.py

How it works:

1. Go through every development request the agent got wrong.
2. Put each mistake in one category: confused intent, wrongly refused, answered an off-topic request, missed entity, extra (spurious) entity, wrong entity type, wrong boundary (too much or too little text), or a value not in the request.
3. Count each category, and keep real examples of each.

Run it (a second):

```
python scripts/explain_stats.py errors
```

```
> python scripts/explain_stats.py errors   (first part)

========================================================================
Error analysis: A1's mistakes on the P1 development requests, by category
  source: reports/dev_analysis.json (written by scripts/run_dev_analysis.py)
========================================================================

spurious entity: 22
    e.g. turn the lights off please -> house_place='lights'
    e.g. it's too dark here -> house_place='here'
    e.g. make coffee now -> time='now'

missed entity: 15
    e.g. make a room darker -> house_place='room'
    e.g. at one p. m. today start the robot vacuum cleaner -> time='one p. m.'
    e.g. at one p. m. today start the robot vacuum cleaner -> date='today'

unsupported value (text not in the request): 7
    e.g. what alarms are upcoming -> date='today'
    e.g. tell me my alarms -> date='today'
    e.g. what time is my wake up alarm set for -> date='tomorrow'

incorrect out-of-scope rejection: 4
    e.g. turn on the t. v. socket when i arrive home
    e.g. i need some more lights
    e.g. start new smart socket

confused intent: 1
```

A1's mistakes by category, with real examples (real output).

What the output means:

- `spurious entity: 22`, e.g. "turn the lights off please -> house_place='lights'": the agent extracted "lights" as a place, which it is not.
- `missed entity: 15`, e.g. "make a room darker": the room was not extracted.
- `incorrect out-of-scope rejection: 4`, e.g. "i need some more lights" was refused as off-topic.

Check it yourself:

1. Run it for P2: `python scripts/explain_stats.py errors --project P2`.
2. Check one example against the data: find "make a room darker" in `data\processed\P1\dev.jsonl`; the correct entity is house_place = room.

Say: "Most of the original agent's mistakes were extra words extracted as entities, which showed us where it was weakest."

Where this comes from, and how it is compared

Who decided it: your project plan, section 8, the error-analysis paragraph: it lists the categories (confused intents, missed entities, wrong boundaries, wrong types, unsupported values, incorrect refusals, missed off-topic, invalid replies and runtime failures).

Why these settings: counting kinds of mistake, not just the number, shows **why** a score is what it is and where to improve. The plan allows looking at development mistakes while improving, but says test-data mistakes may only be inspected after the freeze.

What it is compared with, and how: nothing directly: it explains the scores of the other experiments. Here it was run on A1.

The result: most of A1's mistakes are entity mistakes, mainly extracting words that are not fields (22 per project); intent mistakes are rare (1 on P1, 5 on P2). This is why A1's entity F1 was low, and A2, which picks a design with written descriptions, does much better on entities.

C13. Robustness to messy typing

In one sentence: We checked whether the systems still work when people type carelessly.

Where its files are:

What	Where
The script	scripts\run_robustness.py
Result folders	runs\20261016_0119_P1_A1-robust-typo_0, ...-robust-caps_0, ...-robust-filler_0 (and the same for B1 and B0)
All numbers together	reports\robustness.json
The write-up	reports\DEV_ANALYSIS.md, section 5

How it works:

1. Rewrite every development request three ways, the same way every time (fixed seed):
2. typos: two neighbouring letters swapped ("tunr the lights off please");
3. capitals: "TURN THE LIGHTS OFF PLEASE";
4. filler words: "hey, could you please turn the lights off please, thanks".
5. Score B0, B1 and the agent on each rewritten set, with the same correct answers as before.

Run it (about a minute (answers saved earlier)):

```
python scripts/run_robustness.py
```

What the output means:

- One line per system and version, e.g. `A1 typo intent 0.949`: with typos A1 dropped only from 0.958 to 0.949.

Check it yourself:

1. `python scripts/explain_f1.py --run 20261016_0119_P1_A1-robust-typo_0` shows the requests with their typos ("tunr the lights off please") next to A1's answers, and works out 0.949.
2. Open that folder's `predictions.jsonl`: each line has the rewritten request in `"utt"`.

Say: "The AI systems lose at most about 7 points on messy input, while word matching drops by up to 27."

Where this comes from, and how it is compared

Who decided it: this test is not in the project plan. We added it on 16 October as an extra check, because real users type carelessly and the plan's tests all use clean requests. It is reported as an extra, not as a planned result.

Why these settings: three common kinds of messiness: typos, capital letters and filler words. Each rewrite uses a fixed seed, so it is the same every time, and the typos never touch the entity words, so the correct answers stay valid.

What it is compared with, and how: B0 (word matching), B1 (generic prompt) and A1, each on the clean requests and on each rewritten version, with the same scorer.

The result: the AI systems lose at most about 7 points (A1's worst: P2 with typos, 0.841 to 0.770); word matching drops by up to 27 points (P2 with filler words). Capitals do not affect B0 at all, because it lowercases everything.

> A1 is not more robust than B1 everywhere: on P2 it loses more to typos and capitals. That is in finding F6.

C14. Export and live demo

In one sentence: We saved the chosen pipeline as a package and showed another program can load and use it.

Where its files are:

What	Where
The code	src\runtime\package.py
Scripts	scripts\export_pipeline.py, scripts\demo.py
The P1 package	artifacts\P1\twostage-sim-llm-ff3d548a0af2\ (pipeline.json, examples.jsonl, manifest.json)
Tests for it	tests\test_runtime.py

How it works:

1. Save the chosen design, the Analyzer's descriptions, the examples and the model's fingerprint into three files.
2. Write a fingerprint of each file into manifest.json. If a file is changed later, the fingerprints no longer match and the package refuses to load.
3. Load the package in a fresh program and answer normal, off-topic and invalid requests.
4. These packages hold A1's choices; A2's will be exported at the freeze.

Run it (about 10 seconds each):

```
python scripts/demo.py --project P1
python scripts/demo.py --project P1 --interactive
```

```
> python scripts/demo.py --project P1

Loaded P1 (Home assistant) pipeline twostage-sim-llm, package ff3d548a0af2
Model qwen2.5:3b @ 357c53fb659c | 12 intents, 18 entity types

[normal] 'wake me up at six thirty tomorrow'
  -> ok  intent=alarm_set  time='six thirty', date='tomorrow'  [2 model calls, 1.11s]
[normal] 'turn the bedroom lights off'
  -> ok  intent=iot_hue_lightoff  house_place='bedroom'  [2 model calls, 0.60s]
[out of scope] 'who won the football match last night'
  -> out_of_scope  [1 model calls, 0.29s]
[invalid] ''
  -> invalid_request  error: request is empty  [0 model calls, 0.00s]
[invalid] 'xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx...'
  -> invalid_request  error: request is 600 characters; the limit is 500  [0 model calls, 0.00s]
```

The demo (real output).

What the output means:

- `-> ok intent=alarm_set time='six thirty'`: understood, with the details extracted.
- `-> out_of_scope`: refused, correctly.
- `-> invalid_request`: an empty or too-long request, refused before the AI is even called.

Check it yourself:

1. Open `artifacts\P1\twostage-sim-llm-ff3d548a0af2\manifest.json`: `"files"` lists each file's fingerprint.
2. See the tamper protection work: `python -m pytest tests/test_runtime.py -vv -k modified`. The test edits a package file and checks it is refused (2 passed).
3. In interactive mode, type your own requests, e.g. "dim the kitchen lights", then press Enter on an empty line to stop.

Say: "The agent's choice is saved as a package any program can load, and if anyone edits it, it refuses to load."

Where this comes from, and how it is compared

Who decided it: your project plan, "Runtime and export" (line 423): "Export the ProjectSpec, chosen configuration, prompts, model identifiers (Ollama model name and digest), dependency manifest and version/hash. Provide a simple callable interface first." This is gate G9.

Why these settings: a design is only useful if a real app can load and use it without re-running the agent. The fingerprints stop anyone changing a package unnoticed, and the stored model fingerprint stops it running on a different model than the one it was tested with.

What it is compared with, and how: nothing: this is a working demonstration and an integrity check, tested by `tests\test_runtime.py`.

The result: packages for P1 (twostage-sim-llm) and P2 (schema-fixed-llm) load in a fresh program, handle normal, off-topic and invalid requests, and refuse to load if a file is edited.

> **These packages hold A1's choices.** A2's chosen pipeline (schema-sim-llm on both projects) will be exported at the freeze.

C15. Checking the results themselves

In one sentence: We proved that no recorded number was edited or computed wrongly.

Where its files are:

What	Where
Re-score every run	scripts\reproduce_metrics.py
Independent check with scikit-learn	scripts\verify_with_sklearn.py
Every check at once	scripts\show_evidence.py (saves reports\EVIDENCE_SUMMARY.md)

How it works:

1. For every run folder with a predictions.jsonl, recompute its scores from the saved answers and compare with its metrics.json.
2. Separately, recompute F1, accuracy and the off-topic scores with scikit-learn, without our scorer.
3. Compare to 9 decimal places.

Run it (under a minute in total):

```
python scripts/reproduce_metrics.py
python scripts/verify_with_sklearn.py
python scripts/show_evidence.py
```

```
> python scripts/verify_with_sklearn.py

agrees     20261011_1614_P1_B0_0                    intent F1 0.780, accuracy 0.700, OOS F1 0.510
agrees     20261011_1614_P2_B0_0                    intent F1 0.730, accuracy 0.650, OOS F1 0.510
agrees     20261015_2240_P1_B3-k_0                  intent F1 0.909, accuracy 0.863, OOS F1 0.739
agrees     20261015_2308_P2_B3-k_0                  intent F1 0.833, accuracy 0.775, OOS F1 0.605
agrees     20261024_1622_P1_B2_0                    intent F1 0.858, accuracy 0.775, OOS F1 0.588
agrees     20261024_1630_P2_B2_0                    intent F1 0.861, accuracy 0.825, OOS F1 0.783
agrees     20261024_1654_P1_A1-selected_0           intent F1 0.958, accuracy 0.938, OOS F1 0.909
agrees     20261024_1654_P1_A2-selected_0           intent F1 0.974, accuracy 0.950, OOS F1 0.909
agrees     20261024_1654_P2_A1-selected_0           intent F1 0.841, accuracy 0.800, OOS F1 0.735
agrees     20261024_1654_P2_A2-selected_0           intent F1 0.898, accuracy 0.850, OOS F1 0.776
{'agrees': 44, 'skipped': 1}
Each 'agrees' means scikit-learn computed exactly the number our scorer recorded (5 metrics per run, to 9 decimal places).
```

The independent check (real output).

What the output means:

- `match` / `agrees` on every line: the recorded number is exactly what recomputation gives.
- `not checkable`: that folder has no answers to re-score (for example a timing run). It is not a failure.

Check it yourself:

1. Part A5's explain_f1 does the same three-way check for one run, showing every step: our scorer, scikit-learn and the hand calculation all give 0.9744.
2. Run `python scripts/show_evidence.py` and find "Overall: all checks passed" on the last line.

Say: "Every recorded number can be recomputed from the raw answers, and the standard library scikit-learn gets the same numbers."

Where this comes from, and how it is compared

Who decided it: the re-scoring check comes from your project plan, gate G12 (line 447): "Metrics recompute from saved predictions; every quantitative claim traces to a run." **The scikit-learn check is not in the plan:** we added it on 24 October so the scores are also confirmed by a standard library that has nothing to do with our code.

Why these settings: a recorded number is only trustworthy if it can be recomputed from the raw answers. Comparing to 9 decimal places means even a tiny change would be caught.

What it is compared with, and how: each run's own recorded metrics.json, recomputed in two independent ways.

The result: every checkable run matches both ways (46 at the last count). A test (`tests\test_reproduce_metrics.py`) also proves the check really catches a hand-edited number. The check once found a real gap: B0 and B3 were not saving one field, which was fixed.

Part D. If the teacher says "do it now"

The teacher asks…	Type this	Time	What appears
"Show me a test running."	python scripts/walkthrough.py	5 s	the six steps for three requests
"Is the difference real?"	python scripts/explain_stats.py bootstrap	20 s	the confidence interval, worked out step by step
"How did you measure speed?"	python scripts/explain_stats.py timing	1 s	50 times sorted, median and p95 picked out
"Run all your tests."	python -m pytest tests	15 s	371 passed
"How do you compute F1?"	python scripts/explain_f1.py (Part A5)	5 s	every count and the average, matching metrics.json
"Write a test."	the file from Part B2, then python -m pytest tests/test_my_first.py -vv	5 min	2 passed
"Run your agent."	python scripts/run_a2.py	under 1 min	the AI's rankings, 4 designs tried, the choice
"Run a baseline."	python scripts/run_b0.py --projects P1	seconds	B0 score 0.780
"Is the data clean?"	python scripts/prepare_data.py --check	seconds	sizes and leakage check pass
"Are your numbers real?"	python scripts/verify_with_sklearn.py	seconds	44 agree
"Show me it working."	python scripts/demo.py --project P1 --interactive	live	answers the teacher's own sentences
"Show me everything."	python scripts/show_evidence.py	under 1 min	Overall: all checks passed
"Retrain the published baseline."	explain C6, then open runs\20261015_2240_P1_B3-k_0	show folder	training too long to run live (11 min to 85 min)

> If something fails during a live run, it is almost always because the Ollama app is not open. Open it from the Start menu, wait 10 seconds and run the command again.

Part E. 30-minute rehearsal checklist

Do these once before presenting. Tick each one when it works on your laptop.

- ☐ Open Ollama; in PowerShell run `cd D:\MyClaudeProject\nlp` and `ollama list`.
- ☐ Open `data\processed\P1\dev.jsonl` and a `predictions.jsonl` in Notepad and explain one line of each (Part A3).
- ☐ Run `python scripts/walkthrough.py` and explain each of the six steps out loud (Part A4).
- ☐ Run `python scripts/explain_f1.py`, then count alarm_query yourself in the CSV file (Part A5).
- ☐ Work out the practice example on paper without looking, then check it with `check_f1.py`.
- ☐ Create `tests/test_my_first.py`, run it, break it, fix it.

- ☐ Run `python -m pytest tests` and say what the tests check (Part B3 table).
- ☐ Run `python scripts/run_a2.py` and explain the rankings and the 4 designs tried.
- ☐ For each experiment in Part C, say its idea in one sentence without reading.
- ☐ Run `python scripts/verify_with_sklearn.py` and explain why "agrees" matters.
- ☐ Run the demo in interactive mode and try three sentences of your own.
- ☐ Run `python scripts/show_evidence.py` and find "Overall: all checks passed".

Part F. How to explain the AI use honestly

Your teacher allows AI use. What matters is that you are open about it and understand the work. Something like:

> "We planned the project and decided how it should be tested: which baselines, which data, which scores and which checks. We used Claude Code, an AI coding assistant, to write the code and run the experiments. We checked every result, and we can explain and re-run each test, for example here is a test running step by step. The AI assistance is recorded in docs\contributions.md, as our plan requires."

Then show one or two things live from Part D. Being able to run and explain a test on the spot is the proof that you understand it.

Part G. Build it yourself from scratch

Parts A to F explain what was done, how to run it and how to check it. This part goes one step further: for each experiment, a short script you can read, run and change, that does the same job in the simplest way. If you can explain these, you can show you know how to build the tests yourself.

G0. Before you start: the diy folder

Part C shows how each experiment was run and how to check it. This part shows how you would **build each one yourself**: a short Python script that does the same job in the simplest possible way. These scripts are in the folder `D:\MyClaudeProject\nlp\diy` ("do it yourself"). They are separate from the project's own code, and much shorter, mostly 20 to 50 lines each. Each one was run on 25 October, and the outputs shown are real.

How to open, change and run a script

1. Open the folder in VS Code: in PowerShell, type `cd D:\MyClaudeProject\nlp` and then `code diy`. Or open any file with Notepad.
2. Click a file, for example `c03_b0.py`, to read it. Lines starting with `#` are comments: notes for people, ignored by Python.
3. Run it from PowerShell, in the project folder: `python diy/c03_b0.py`.
4. Change something, save (Ctrl+S) and run it again. For example, change `0.35` to `0.5` in c03 and see the score change.
5. To write your own from nothing: create a new file in the diy folder (in VS Code: File → New File), name it something like `my_test.py`, type the code, save, and run it the same way.

The shared helper file: diy\helpers.py

Three small functions that every script uses, so they are written once:

```
"""Three small helpers used by every "do it yourself" script.

These are deliberately simple, so you can read and explain every line.
They do the same jobs as the project's own code, in the plainest way.
"""

import json
import urllib.request

# The AI model runs inside the Ollama app on this laptop, at this address.
OLLAMA_URL = "http://127.0.0.1:11434/api/generate"
MODEL = "qwen2.5:3b"


def load_jsonl(path):
    """Read a .jsonl file: one JSON record per line. Returns a list of dictionaries."""
    with open(path, encoding="utf-8") as f:
        return [json.loads(line) for line in f if line.strip()]


def ask_model(prompt):
    """Send one prompt to the AI model and return its reply as text.

    temperature 0 and seed 0 make the model give the same answer every time;
    format "json" asks it to reply in JSON.
    """
    request_body = {
        "model": MODEL,
        "prompt": prompt,
        "stream": False,
        "format": "json",
        "options": {"temperature": 0, "seed": 0},
    }
    request = urllib.request.Request(
        OLLAMA_URL,
        data=json.dumps(request_body).encode("utf-8"),
        headers={"Content-Type": "application/json"},
    )
    with urllib.request.urlopen(request, timeout=120) as response:
        return json.loads(response.read())["response"]


def read_intent(reply_text):
    """Get the intent out of the model's JSON reply. Returns None if the reply is broken."""
    try:
        return json.loads(reply_text).get("intent")
    except (json.JSONDecodeError, AttributeError):
        return None
```

Function	What it does
load_jsonl(path)	opens a .jsonl file and returns its lines as a list of records (the same format as File 1 in Part A3)
ask_model(prompt)	sends a prompt to the AI model running in the Ollama app on this laptop and returns its reply. It sends a small message to the address http://127.0.0.1:11434 with the prompt and the settings. temperature 0 means "always pick the most likely answer", so replies are repeatable
read_intent(text)	reads the intent out of the model's JSON reply, or gives None if the reply is broken

The scripts for C7 to C14 also use `diy\mini_pipeline.py`: the C4 pipeline turned into functions, with two switches (which examples to show, and how to refuse off-topic requests). That gives 4 small designs, a mini version of the project's 12, so you can build a mini agent and mini ablations on top of it.

G1. Build it yourself: The data and the splits (goes with C1)

Build P1's split yourself, straight from the original MASSIVE file.

The file: `diy\c01_make_split.py` (open it with `code diy\c01_make_split.py`):

```
"""C1 yourself: build a project's data split straight from the original MASSIVE file.

Run from the project folder:  python diy/c01_make_split.py
"""

import json
import random
from collections import Counter

# 1. Read the original dataset: one request per line, labelled by people.
records = []
with open("data/raw/1.1/data/en-US.jsonl", encoding="utf-8") as f:
    for line in f:
        records.append(json.loads(line))
print("requests in MASSIVE (English):", len(records))

# 2. Keep only the domains our home-assistant project handles.
project_domains = ["alarm", "iot"]
mine = [r for r in records if r["scenario"] in project_domains]
print("requests about alarms and smart-home devices:", len(mine))

# 3. MASSIVE already says which requests are for training, development and testing.
train = [r for r in mine if r["partition"] == "train"]
dev = [r for r in mine if r["partition"] == "dev"]
test = [r for r in mine if r["partition"] == "test"]
print("official split sizes: train", len(train), "| dev", len(dev), "| test", len(test))

# 4. Draw 10 examples per intent from the training part. A fixed seed means
#    the same "random" draw every time anyone runs this.
random.seed(42)
examples = []
for intent in sorted({r["intent"] for r in train}):
    candidates = [r for r in train if r["intent"] == intent]
    examples += random.sample(candidates, min(10, len(candidates)))
print("examples drawn:", len(examples), "(10 per intent)")

# 5. Draw 80 development requests the same way.
dev_sample = random.sample(dev, 80)
print("development requests drawn:", len(dev_sample))
print("intents in the development sample:", dict(Counter(r["intent"] for r in dev_sample)))

# 6. The leakage check: no request may be in two places.
example_ids = {r["id"] for r in examples}
dev_ids = {r["id"] for r in dev_sample}
print("requests in both examples and development (must be 0):", len(example_ids & dev_ids))

# 7. Save them, one request per line.
with open("diy/my_dev.jsonl", "w", encoding="utf-8") as f:
    for r in dev_sample:
        f.write(json.dumps({"id": r["id"], "utt": r["utt"], "intent": r["intent"]}) + "\n")
print("saved diy/my_dev.jsonl")
```

The key lines:

Line	What it does
`records.append(json.loads(line))`	reads the dataset one request at a time: all 16,521
`r["scenario"] in project_domains`	keeps only alarm and smart-home requests
`random.seed(42)`	fixes the random draw, so it comes out the same every time
`random.sample(candidates, 10)`	draws 10 examples of each intent at random
`example_ids & dev_ids`	the leakage check: requests that appear in both sets (must be none)

Run it:

```
python diy/c01_make_split.py
```

What you should see (real output):

```
requests in MASSIVE (English): 16521
requests about alarms and smart-home devices: 1657
official split sizes: train 1159 | dev 182 | test 316
examples drawn: 120 (10 per intent)
development requests drawn: 80
intents in the development sample: {'iot_hue_lightup': 6, 'alarm_set': 11, 'alarm_remove': 8,
'alarm_query': 8, 'iot_cleaning': 9, 'iot_wemo_on': 1, 'iot_hue_lightdim': 9, 'iot_coffee':
7, 'iot_hue_lightchange': 7, 'iot_hue_lightoff': 10, 'iot_wemo_off': 2, 'iot_hue_lighton': 2}
requests in both examples and development (must be 0): 0
saved diy/my_dev.jsonl
```

What to notice: It finds the same 1,159 training requests for P1 that B3-full trained on. Its random draw differs from the project's (different seed, and the project also balances intents in the development set), but the method is the same.

G2. Build it yourself: Model speed and repeatability (goes with C2)

Time the model on 10 requests and check it gives the same answers twice.

The file: `diy\c02_speed.py` (open it with `code diy\c02_speed.py`):

```
"""C2 yourself: is the AI model fast, and does it give the same answer twice?

Needs the Ollama app open.  Run:  python diy/c02_speed.py
"""

import statistics
import time

from helpers import ask_model, load_jsonl

requests = load_jsonl("data/processed/P1/dev.jsonl")[:10]   # the first 10 development
requests

first_answers, times = [], []
for r in requests:
    prompt = f'Which smart-home action is this? Reply as JSON {{"intent": "..."}}. Request:
{r["utt"]}'
    start = time.perf_counter()                # start the stopwatch
    reply = ask_model(prompt)
    times.append(time.perf_counter() - start)   # stop it: seconds this request took
    first_answers.append(reply)

# Ask exactly the same 10 again and compare the answers word for word.
second_answers = [ask_model(f'Which smart-home action is this? Reply as JSON {{"intent":
"..."}}. Request: {r["utt"]}')
                  for r in requests]
same = sum(a == b for a, b in zip(first_answers, second_answers))

print("seconds per request:", [round(t, 2) for t in times])
print("median:", round(statistics.median(times), 2), "s  (the first request is slower: it
wakes the model up)")
print("identical answers on repeat:", same, "of", len(requests))
```

The key lines:

Line	What it does
`time.perf_counter()`	a stopwatch: read it before and after a request, and subtract
`a == b for a, b in zip(...)`	compares the first and second answers pair by pair

Run it:

```
python diy/c02_speed.py
```

What you should see (real output):

```
seconds per request: [0.54, 0.41, 0.2, 0.35, 0.23, 0.25, 0.17, 0.18, 0.53, 0.31]
median: 0.28 s  (the first request is slower: it wakes the model up)
identical answers on repeat: 10 of 10
```

What to notice: The first request is slow (it wakes the model up). That is why the real timing test throws away warm-up requests. These prompts are shorter than the project's, so they are faster.

G3. Build it yourself: B0: word matching, no AI (goes with C3)

Build the word-matching baseline yourself: about 20 lines, no AI model.

The file: `diy\c03_b0.py` (open it with `code diy\c03_b0.py`):

```
"""C3 yourself: build the word-matching baseline (B0) from scratch.

No AI model needed.  Run:  python diy/c03_b0.py
"""

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, f1_score

from helpers import load_jsonl

# 1. Load the 120 labelled examples (to learn from) and the 100 development requests (to test
on).
examples = load_jsonl("data/processed/P1/kshot.jsonl")
dev = load_jsonl("data/processed/P1/dev.jsonl")
in_scope = [r for r in dev if r["in_scope"]]      # the 80 that have a real intent

# 2. Turn sentences into numbers: each word gets a weight (TF-IDF).
#    ngram_range=(1, 2) also counts pairs of words, like "turn off".
vectorizer = TfidfVectorizer(ngram_range=(1, 2), sublinear_tf=True)
X_train = vectorizer.fit_transform([r["utt"] for r in examples])
y_train = [r["true_intent"] for r in examples]

# 3. Train a classifier that learns which word weights point to which intent.
classifier = LogisticRegression(max_iter=2000, C=10.0, random_state=0)
classifier.fit(X_train, y_train)

# 4. Predict the intent of every development request.
X_dev = vectorizer.transform([r["utt"] for r in in_scope])
predicted = classifier.predict(X_dev)

# 5. Refuse a request when the classifier is unsure (its top probability is low).
#    The project chose this cut-off on development data; here we just pick 0.35.
confidence = classifier.predict_proba(X_dev).max(axis=1)
predicted = ["REFUSED" if c < 0.35 else p for p, c in zip(predicted, confidence)]

# 6. Score it.
correct = [r["true_intent"] for r in in_scope]
labels = sorted(set(correct))
print("first 5:", list(zip([r["utt"] for r in in_scope[:5]], predicted[:5])))
print("accuracy:", round(accuracy_score(correct, predicted), 3))
print("intent macro-F1:", round(f1_score(correct, predicted, labels=labels, average="macro",
zero_division=0), 3))
```

The key lines:

Line	What it does
`TfidfVectorizer(...)`	turns each sentence into word weights
`classifier.fit(X_train, y_train)`	the training: learns which words point to which intent
`classifier.predict(X_dev)`	guesses the intent of each development request
`predict_proba(X_dev).max(axis=1)`	how sure it is about its guess (0 to 1)
`"REFUSED" if c < 0.35`	refuses when it is not sure enough
`f1_score(..., average="macro")`	scores it, exactly as in Part A5

Run it:

```
python diy/c03_b0.py
```

What you should see (real output):

```
first 5: [('turn the lights off please', 'iot_hue_lightoff'), ('make a room darker',
'REFUSED'), ('set an alarm at six in the morning', 'alarm_set'), ("it's too dark here",
'iot_hue_lighton'), ('turn off the smart plug', 'iot_wemo_off')]
accuracy: 0.688
intent macro-F1: 0.771
```

What to notice: It scores 0.771; the project's B0 scores 0.780. The difference comes from the refusal rule: the project chose its cut-off carefully on development data and uses a slightly different measure of "sure", while this version just uses 0.35. Try other cut-offs and watch the score move.

G4. Build it yourself: B1: one generic prompt (goes with C4)

Build a complete one-prompt AI pipeline yourself, and score it on all 100 requests.

The file: `diy\c04_b1.py` (open it with `code diy\c04_b1.py`):

```
"""C4 yourself: build a one-prompt AI pipeline (like B1) and score it.

Needs the Ollama app open.  Run:  python diy/c04_b1.py          (all 100 requests, about a
minute)
                                  python diy/c04_b1.py 20       (only the first 20)
"""

import sys

from sklearn.metrics import accuracy_score, f1_score

from helpers import ask_model, load_jsonl, read_intent

examples = load_jsonl("data/processed/P1/kshot.jsonl")
dev = load_jsonl("data/processed/P1/dev.jsonl")
limit = int(sys.argv[1]) if len(sys.argv) > 1 else len(dev)
dev = dev[:limit]

# 1. The list of intents the app supports, and one worked example for each.
intents = sorted({r["true_intent"] for r in examples})
one_example_each = [next(r for r in examples if r["true_intent"] == i) for i in intents]


# 2. Build the prompt: instructions, the allowed intents, the examples, then the request.
def build_prompt(request):
    lines = ["Label this request for a home assistant. Reply with JSON only.", "", "Allowed
intents:"]
    lines += [f"- {i}" for i in intents]
    lines += ["- none_of_the_above  (use this if the request is about anything else)", "",
"Examples:"]
    lines += [f'Request: {e["utt"]}\nAnswer: {{"intent": "{e["true_intent"]}"}}' for e in
one_example_each]
    lines += ["", f"Request: {request}", "Answer:"]
    return "\n".join(lines)


# 3. Ask the model about every development request and read its answer.
correct, answered = [], []
for number, r in enumerate(dev, 1):
    intent = read_intent(ask_model(build_prompt(r["utt"])))
    correct.append(r["true_intent"] if r["in_scope"] else "none_of_the_above")
    answered.append(intent or "BROKEN REPLY")
    if number <= 5:
        print(f'{r["utt"]!r:45} correct: {correct[-1]:20} model: {answered[-1]}')

# 4. Score: intent F1 over the in-scope requests (refusing one counts as wrong),
#    and how many off-topic requests were correctly refused.
pairs = [(c, a) for c, a in zip(correct, answered) if c != "none_of_the_above"]
gold, pred = [c for c, _ in pairs], [a for _, a in pairs]
off_topic = [(c, a) for c, a in zip(correct, answered) if c == "none_of_the_above"]
print("\nrequests:", len(dev))
print("intent accuracy:", round(accuracy_score(gold, pred), 3))
print("intent macro-F1:", round(f1_score(gold, pred, labels=intents, average="macro",
zero_division=0), 3))
print("off-topic requests correctly refused:", sum(a == c for c, a in off_topic), "of",
len(off_topic))
```

The key lines:

Line	What it does
`one_example_each = ...`	picks one worked example for each intent
`def build_prompt(request)`	writes the prompt: instructions, the allowed intents, the examples, the request
`ask_model(build_prompt(r["utt"]))`	asks the AI model about one request
`read_intent(...)`	reads the intent from its JSON reply
`f1_score(gold, pred, ...)`	scores the 80 in-scope requests; refused ones count as wrong

Run it:

```
python diy/c04_b1.py
```

What you should see (real output):

```
'turn the lights off please'                correct: iot_hue_lightoff     model:
iot_hue_lightoff
'make a room darker'                        correct: iot_hue_lightdim     model:
iot_hue_lightdim
'set an alarm at six in the morning'        correct: alarm_set            model: alarm_set
"it's too dark here"                        correct: iot_hue_lighton      model:
iot_hue_lightup
'turn off the smart plug'                   correct: iot_wemo_off         model:
iot_wemo_off

requests: 100
intent accuracy: 0.9
intent macro-F1: 0.927
off-topic requests correctly refused: 20 of 20
```

What to notice: This 40-line script gives intent macro-F1 0.927 and refuses all 20 off-topic requests, the same as the project's B1. It takes about 20 seconds.

G5. Build it yourself: B2: a person configuring by hand (goes with C5)

Do it yourself: B2 is already a by-hand job. To do it without the worksheet script: read `configs\descriptions\P1.md`, pick one of the 12 designs, try it with `python scripts/walkthrough.py --design <your design> --n 10`, look at which requests it gets wrong, try another design, and time yourself from start to finish.

G6. Build it yourself: B3: the published baseline (XLM-R) (goes with C6)

Fine-tune XLM-R yourself. This version learns intents only (the project's B3 also learns entities). It runs for about 8 to 9 minutes on the laptop's processor; no Ollama needed.

The file: `diy\c06_finetune.py` (open it with `code diy\c06_finetune.py`):

```
"""C6 yourself: fine-tune the published baseline model (XLM-R) to recognise intents.

The project's B3 also learns entities; this simpler version learns intents only,
which is enough to see how fine-tuning works. No Ollama needed, but it trains on
the laptop's processor: about 5 to 10 minutes.  Run:  python diy/c06_finetune.py
"""

import os

os.environ["HF_HUB_OFFLINE"] = "1"   # the model is already downloaded; don't go online

import torch
from sklearn.metrics import accuracy_score, f1_score
from transformers import AutoModelForSequenceClassification, AutoTokenizer

from helpers import load_jsonl

torch.manual_seed(0)                                  # same result every run

# 1. The data: 120 labelled examples to learn from, 80 in-scope development requests to test
on.
examples = load_jsonl("data/processed/P1/kshot.jsonl")
dev = [r for r in load_jsonl("data/processed/P1/dev.jsonl") if r["in_scope"]]
intents = sorted({r["true_intent"] for r in examples})
number_of = {name: i for i, name in enumerate(intents)}   # the model works with numbers, not
names

# 2. Load XLM-R and add a small "intent" layer on top, with one output per intent.
tokenizer = AutoTokenizer.from_pretrained("xlm-roberta-base")
model = AutoModelForSequenceClassification.from_pretrained("xlm-roberta-base",
num_labels=len(intents))


def to_numbers(sentences):
    """Split sentences into word pieces and turn them into numbers the model reads."""
    return tokenizer(sentences, padding=True, truncation=True, max_length=48,
return_tensors="pt")


# 3. Fine-tune: show the examples again and again; after each batch, nudge the model's
#    internal numbers so its guesses get closer to the correct intents.
optimizer = torch.optim.AdamW(model.parameters(), lr=3e-5)
model.train()
for epoch in range(30):                               # 30 passes over the examples
    order = torch.randperm(len(examples)).tolist()
    for start in range(0, len(order), 16):            # 16 examples at a time
        batch = [examples[i] for i in order[start:start + 16]]
        inputs = to_numbers([r["utt"] for r in batch])
        labels = torch.tensor([number_of[r["true_intent"]] for r in batch])
        loss = model(**inputs, labels=labels).loss    # how wrong the model is on this batch
        loss.backward()                               # work out which way to nudge each
number
        optimizer.step()                              # nudge them
        optimizer.zero_grad()
    if (epoch + 1) % 5 == 0:
        print(f"pass {epoch + 1}/30, how wrong (loss): {loss.item():.3f}")

# 4. Test it on the development requests it has never seen.
model.eval()
```

```
with torch.no_grad():
    scores = model(**to_numbers([r["utt"] for r in dev])).logits
predicted = [intents[i] for i in scores.argmax(dim=1).tolist()]
correct = [r["true_intent"] for r in dev]
print("accuracy:", round(accuracy_score(correct, predicted), 3))
print("intent macro-F1:", round(f1_score(correct, predicted, labels=intents, average="macro",
zero_division=0), 3))
```

The key lines:

Line	What it does
`AutoModelForSequenceClassification`	loads XLM-R and adds a small layer with one output per intent
`for epoch in range(30)`	30 passes over the 120 examples
`loss = model(...).loss`	how wrong the model is on the current batch
`loss.backward() / optimizer.step()`	work out how to change the model's numbers, then change them a little
`scores.argmax(dim=1)`	for each request, the intent with the highest score

Run it:

```
python diy/c06_finetune.py
```

What you should see (real output):

```
pass 5/30, how wrong (loss): 2.000
pass 10/30, how wrong (loss): 1.298
pass 15/30, how wrong (loss): 0.680
pass 20/30, how wrong (loss): 0.148
pass 25/30, how wrong (loss): 0.083
pass 30/30, how wrong (loss): 0.058
accuracy: 0.938
intent macro-F1: 0.94
```

What to notice: The "how wrong" number falls from 2.0 to 0.06 as it learns. It reaches 0.940, higher than the project's B3-k (0.909), because this version never refuses anything, so it can never wrongly refuse a valid request; the project's B3 also has to reject off-topic ones.

G7. Build it yourself: B4: random search (goes with C7)

Run random search yourself, using the 12 design scores the project measured.

The file: `diy\c07_random_search.py` (open it with `code diy\c07_random_search.py`):

```
"""C7 yourself: random search. Pick 4 of the 12 designs at random, keep the best, repeat for
5 seeds.

Uses the scores of all 12 designs that the project already measured, so it needs no model.
Run:  python diy/c07_random_search.py
"""

import json
import random
from math import comb
from pathlib import Path

# 1. The development score of each of the 12 designs on P1 (measured by B4-exhaustive).
folder = sorted(Path("runs").glob("*_P1_B4-exhaustive_0"))[-1]
ranking = json.loads((folder / "config.json").read_text(encoding="utf-8"))["selection"]
["ranking"]
score_of = {row["config_id"]: row["intent_macro_f1"] for row in ranking}
designs = sorted(score_of)
for d in designs:
    print(f"  {d:22} {score_of[d]:.3f}")

# 2. Random search: draw 4 designs, keep the best of those 4. Do it with 5 different seeds.
winners = []
for seed in range(5):
    random.seed(seed)
    drawn = random.sample(designs, 4)
    best = max(drawn, key=lambda d: score_of[d])
    winners.append(score_of[best])
    print(f"seed {seed}: drew {drawn} -> kept {best} ({score_of[best]:.3f})")

print("average of the 5 winners:", round(sum(winners) / len(winners), 3))
print("best possible (all 12):", round(max(score_of.values()), 3))

# 3. Why random search does well here: the chance that 4 random picks include the single best
design.
chance = 1 - comb(11, 4) / comb(12, 4)
print(f"chance 4 random picks include the best design: {chance:.0%}")
good = [d for d in designs if score_of[d] >= max(score_of.values()) - 0.06]
print(f"designs within 0.06 of the best: {len(good)} of 12, so a random 4 usually contains a
good one")
```

The key lines:

Line	What it does
`random.sample(designs, 4)`	draws 4 designs at random
`max(drawn, key=...)`	keeps the best of the 4
`for seed in range(5)`	repeats with 5 different seeds
`1 - comb(11, 4) / comb(12, 4)`	the chance that 4 random picks include the single best design

Run it:

```
python diy/c07_random_search.py
```

What you should see (real output):

```
joint-fixed-llm        0.927
  joint-fixed-tfidf      0.849
  joint-sim-llm          0.951
  joint-sim-tfidf        0.862
  schema-fixed-llm       0.950
  schema-fixed-tfidf     0.845
  schema-sim-llm         0.974
  schema-sim-tfidf       0.869
  twostage-fixed-llm     0.901
  twostage-fixed-tfidf   0.839
  twostage-sim-llm       0.958
  twostage-sim-tfidf     0.858
seed 0: drew ['schema-sim-llm', 'twostage-sim-tfidf', 'joint-fixed-llm', 'schema-fixed-llm']
-> kept schema-sim-llm (0.974)
seed 1: drew ['joint-sim-llm', 'twostage-fixed-tfidf', 'joint-fixed-tfidf', 'schema-fixed-
llm'] -> kept joint-sim-llm (0.951)
seed 2: drew ['joint-fixed-llm', 'joint-fixed-tfidf', 'twostage-sim-llm', 'schema-fixed-
tfidf'] -> kept twostage-sim-llm (0.958)
seed 3: drew ['joint-sim-tfidf', 'twostage-fixed-tfidf', 'twostage-fixed-llm', 'joint-sim-
llm'] -> kept joint-sim-llm (0.951)
seed 4: drew ['joint-sim-tfidf', 'schema-fixed-llm', 'joint-fixed-tfidf', 'schema-sim-llm'] -
> kept schema-sim-llm (0.974)
average of the 5 winners: 0.962
best possible (all 12): 0.974
chance 4 random picks include the best design: 33%
designs within 0.06 of the best: 5 of 12, so a random 4 usually contains a good one
```

What to notice: It averages 0.962 (the project's 0.959 used different random draws). There is a 33% chance of drawing the best design, and 5 of the 12 designs are within 0.06 of the best, which is why random search is hard to beat here.

G8. Build it yourself: A1 and A2: the agent (goes with C8)

Build a mini agent yourself: the AI ranks 4 designs, two are tested, it sees the scores, picks one more, and the best is kept. About a minute.

The file: `diy\c08_mini_agent.py` (open it with `code diy\c08_mini_agent.py`):

```
"""C8 yourself: a mini agent that chooses its own pipeline design.

The AI ranks the 4 designs of mini_pipeline.py, the two it likes best are
tested, it sees their scores and picks one more, and the best of the three wins.
This is the same idea as the project's Architect, at a small scale.

Needs the Ollama app open. About a minute.  Run:  python diy/c08_mini_agent.py
"""

import json

from helpers import ask_model
from mini_pipeline import DESIGNS, DEV, evaluate

requests = DEV[:40]                        # a smaller set keeps it quick
names = [f"{e}-{r}" for e, r in DESIGNS]   # e.g. "similar-model"
EXPLAIN = (
    "examples=fixed: one example per intent, always the same. "
    "examples=similar: the 6 examples most similar to each request. "
    "refusal=model: the model may answer none_of_the_above. "
    "refusal=cutoff: requests unlike every example are refused before asking the model."
)

# 1. The agent plans: ask the AI to rank the 4 designs for this project, with reasons.
plan = ask_model(
    f"You design a pipeline for a home assistant that sets alarms and controls lights and
plugs. "
    f"Users phrase things many ways; off-topic requests must be refused. Designs: {names}.
{EXPLAIN} "
    'Rank all 4 designs from best to worst. Reply as JSON: {"ranking": [...], "why": "..."}'
)
plan = json.loads(plan)
print("AI's ranking, as it wrote it:", plan.get("ranking"))
ranking = [d for d in plan.get("ranking", []) if d in names] or names   # ignore anything
invented
print("after removing names that are not real designs:", ranking)
print("AI's reason:", plan.get("why"))

# 2. Test the two designs it ranked highest.
scores = {}
for name in ranking[:2]:
    examples, refusal = name.split("-")
    scores[name], _ = evaluate(requests, examples=examples, refusal=refusal)
    print(f"tested {name}: intent macro-F1 {scores[name]:.3f}")

# 3. The agent revises: show it the scores and let it pick one more design to test.
remaining = [n for n in names if n not in scores]
revision = json.loads(ask_model(
    f"You tested these designs: {json.dumps({k: round(v, 3) for k, v in scores.items()})}.
{EXPLAIN} "
    f'Which ONE of {remaining} should be tested next? Reply as JSON: {{"next": "...", "why":
"..."}}'
))
next_design = revision.get("next") if revision.get("next") in remaining else remaining[0]
examples, refusal = next_design.split("-")
scores[next_design], _ = evaluate(requests, examples=examples, refusal=refusal)
print(f"AI chose next: {next_design} ({revision.get('why')})")
print(f"tested {next_design}: intent macro-F1 {scores[next_design]:.3f}")
```

```
# 4. Select: keep the best-scoring design.
best = max(scores, key=scores.get)
print(f"\nselected: {best} with intent macro-F1 {scores[best]:.3f}")
```

The key lines:

Line	What it does
`ask_model(... Rank all 4 designs ...)`	the agent plans: it ranks the designs and gives a reason
`if d in names`	ignores any design name the AI invents; only real designs are tested
`evaluate(requests, ...)`	tests a design on 40 development requests
`ask_model(... tested these designs ...)`	the agent revises: it sees the scores and picks the next design
`max(scores, key=scores.get)`	keeps the best-scoring design

Run it:

```
python diy/c08_mini_agent.py
```

What you should see (real output):

```
AI's ranking, as it wrote it: ['similar-cutoff', 'fixed-cutoff', 'fixed-model',
'refusal=model']
after removing names that are not real designs: ['similar-cutoff', 'fixed-cutoff', 'fixed-
model']
AI's reason: The 'similar-cutoff' design is the best because it allows for a more nuanced
understanding of user requests by considering the similarity to previous examples. This
design can handle variations in user phrasing while still ensuring that off-topic requests
are refused early. The 'fixed-cutoff' design is next, as it uses a fixed set of examples but
still has the advantage of being able to handle some variations. The 'fixed-model' design is
less flexible and may struggle with new or varied requests. Lastly, the 'refusal=model'
design is the worst because it refuses requests without even giving the model a chance to
process them, which can lead to many unnecessary refusals and missed opportunities for
understanding user intent.
tested similar-cutoff: intent macro-F1 0.872
tested fixed-cutoff: intent macro-F1 0.957
AI chose next: fixed-model (The previous tests showed that the 'fixed-cutoff' design
performed better with a cutoff of 0.957, which is closer to the cutoff of 0.957 used in the
fixed-cutoff design. This suggests that the fixed-model might be more aligned with the
model's performance on the fixed-cutoff design, making it a good candidate for the next
test.)
tested fixed-model: intent macro-F1 0.858

selected: fixed-cutoff with intent macro-F1 0.957
```

What to notice: The AI listed "refusal=model" as a fourth design. That is not a design name, so the script removed it, exactly as the project's Architect does: you can see both lines in the output. The ranking and the scores were the same in both runs, but the wording of the AI's reason differed between runs: long written answers can vary slightly even with temperature 0.

G9. Build it yourself: Ablations (goes with C9)

Run an ablation yourself: switch off one part of the pipeline at a time.

The file: `diy\c09_ablation.py` (open it with `code diy\c09_ablation.py`):

```
"""C9 yourself: an ablation. Remove one part of the pipeline and see how much the score
drops.

Needs the Ollama app open. About a minute.  Run:  python diy/c09_ablation.py
"""

from mini_pipeline import DEV, evaluate

requests = DEV  # all 100 development requests

# 1. The full pipeline: fixed examples in the prompt, the model may refuse.
full, _ = evaluate(requests, examples="fixed", refusal="model")
print(f"full pipeline:            intent macro-F1 {full:.3f}")

# 2. Remove the examples from the prompt; everything else stays the same.
no_examples, _ = evaluate(requests, examples="fixed", refusal="model", show_examples=False)
print(f"without examples:         intent macro-F1 {no_examples:.3f}   (change {no_examples -
full:+.3f})")

# 3. Swap the model's own refusal for the simple similarity cut-off.
cutoff, _ = evaluate(requests, examples="fixed", refusal="cutoff")
print(f"cut-off instead of model: intent macro-F1 {cutoff:.3f}   (change {cutoff -
full:+.3f})")

print("\nA part that matters makes the score drop when it is removed.")
```

The key lines:

Line	What it does
`show_examples=False`	removes the worked examples from the prompt
`refusal="cutoff"`	replaces the model's own refusal with the similarity cut-off

Run it:

```
python diy/c09_ablation.py
```

What you should see (real output):

```
full pipeline:            intent macro-F1 0.927
without examples:         intent macro-F1 0.899   (change -0.027)
cut-off instead of model: intent macro-F1 0.925   (change -0.002)

A part that matters makes the score drop when it is removed.
```

What to notice: Removing the examples costs 2.7 points, so they matter. Swapping the refusal method barely changes anything here. That is the whole idea of an ablation.

G10. Build it yourself: Confidence intervals (bootstrap) (goes with C10)

Compute a bootstrap confidence interval yourself, from the answers the project saved.

The file: `diy\c10_bootstrap.py` (open it with `code diy\c10_bootstrap.py`):

```
"""C10 yourself: a bootstrap confidence interval for "is A1 really better than B1?".

Uses the answers the project saved for A1 and B1 on P1. No model needed.
Run:  python diy/c10_bootstrap.py
"""

import random
from pathlib import Path

from sklearn.metrics import f1_score

from helpers import load_jsonl


def intent_answers(run_pattern):
    """Each in-scope request's correct intent and the system's answer, from a saved run."""
    folder = sorted(Path("runs").glob(run_pattern))[-1]
    rows = [r for r in load_jsonl(folder / "predictions.jsonl") if r["gold_in_scope"]]
    said = [r["predicted_intent"] if r["status"] == "ok" else "REFUSED" for r in rows]
    return [r["gold_intent"] for r in rows], said


gold, a1 = intent_answers("*_P1_A1-selected_0")
_, b1 = intent_answers("*_P1_B1_0")
labels = sorted(set(gold))


def macro_f1(indices, answers):
    return f1_score([gold[i] for i in indices], [answers[i] for i in indices],
                    labels=labels, average="macro", zero_division=0)


everyone = list(range(len(gold)))
print(f"real difference (A1 - B1): {macro_f1(everyone, a1) - macro_f1(everyone, b1):+.3f}")

# 1000 times: draw a pretend set of the same size, with repeats allowed, and score both on
it.
random.seed(0)
differences = []
for _ in range(1000):
    drawn = [random.randrange(len(gold)) for _ in everyone]
    differences.append(macro_f1(drawn, a1) - macro_f1(drawn, b1))

differences.sort()
low, high = differences[25], differences[974]          # cut 2.5% off each end: the middle
95%
print(f"95% confidence interval: [{low:+.3f}, {high:+.3f}]")
print(f"A1 better in {sum(d > 0 for d in differences) / 10:.0f}% of the 1000 draws")
print("includes 0, so it could be luck" if low <= 0 <= high else "entirely above 0, so it is
unlikely to be luck")
```

The key lines:

Line	What it does
`random.randrange(len(gold))`	draws one request number; repeating this 80 times makes a pretend set
`differences.append(...)`	the A1 minus B1 difference on that pretend set
`differences[25]`, `differences[974]`	after sorting the 1,000 differences, cut 2.5% off each end

Run it:

```
python diy/c10_bootstrap.py
```

What you should see (real output):

```
real difference (A1 - B1): +0.032
95% confidence interval: [-0.042, +0.118]
A1 better in 82% of the 1000 draws
includes 0, so it could be luck
```

What to notice: It gives [-0.042, +0.118]; the project's gives [-0.040, +0.121]. They differ slightly because the random draws differ, but both include 0, so both reach the same conclusion.

G11. Build it yourself: Real speed (timing) (goes with C11)

Measure speed yourself: warm up, time 30 requests, sort, pick the median and p95.

The file: `diy\c11_timing.py` (open it with `code diy\c11_timing.py`):

```
"""C11 yourself: measure the pipeline's real speed, median and p95.

Needs the Ollama app open. About 20 seconds.  Run:  python diy/c11_timing.py
"""

import time

from mini_pipeline import DEV, answer

# 1. Warm up: the first requests wake the model up and are slower, so they are not counted.
for r in DEV[:3]:
    answer(r["utt"])

# 2. Time 30 requests with a stopwatch.
times = []
for r in DEV[3:33]:
    start = time.perf_counter()
    answer(r["utt"])
    times.append(time.perf_counter() - start)

# 3. Sort them. The median is the middle one; p95 is the one 95% of requests are faster than.
times.sort()
print("sorted times (s):", [round(t, 2) for t in times])
print(f"median: {times[len(times) // 2]:.2f} s")
print(f"p95:    {times[min(len(times) - 1, int(0.95 * len(times)))]:.2f} s")
```

The key lines:

Line	What it does
`for r in DEV[:3]: answer(...)`	warm-up requests that are not counted
`times.sort()`	sorts the times from fastest to slowest
`times[len(times) // 2]`	the middle one: the median

Run it:

```
python diy/c11_timing.py
```

What you should see (real output):

```
sorted times (s): [0.14, 0.15, 0.15, 0.15, 0.16, 0.16, 0.16, 0.16, 0.16, 0.16, 0.16, 0.16,
0.16, 0.17, 0.17, 0.17, 0.18, 0.18, 0.19, 0.19, 0.19, 0.19, 0.19, 0.19, 0.19, 0.19, 0.19,
0.2, 0.2, 0.2]
median: 0.17 s
p95:    0.20 s
```

What to notice: This one-call pipeline answered in about 0.2 to 0.3 s in our runs (0.17 s median in the run shown, 0.27 s in an earlier one). The project's A1 on P1 uses two calls per request and longer prompts, so it takes longer (0.77 s).

G12. Build it yourself: Error analysis (goes with C12)

Sort a system's mistakes into kinds yourself, using A1's saved answers on P2.

The file: `diy\c12_errors.py` (open it with `code diy\c12_errors.py`):

```
"""C12 yourself: error analysis. Sort a system's mistakes into kinds and count them.

Uses the answers the project saved for A1 on P2. No model needed.
Run:  python diy/c12_errors.py
"""

from collections import Counter
from pathlib import Path

from helpers import load_jsonl

folder = sorted(Path("runs").glob("*_P2_A1-selected_0"))[-1]
rows = load_jsonl(folder / "predictions.jsonl")
requests = {r["case_id"]: r["utt"] for r in load_jsonl("data/processed/P2/dev.jsonl")}

kinds = Counter()
mix_ups = Counter()
for r in rows:
    request = requests[r["case_id"]]
    if not r["gold_in_scope"]:
        if r["status"] == "ok":                 # should have been refused, but was answered
            kinds["answered an off-topic request"] += 1
            print(f"answered off-topic: {request!r} -> {r['predicted_intent']}")
    elif r["status"] == "oos":                  # should have been answered, but was refused
        kinds["wrongly refused"] += 1
        print(f"wrongly refused:    {request!r} (correct: {r['gold_intent']})")
    elif r["status"] != "ok":
        kinds["broken reply"] += 1
    elif r["predicted_intent"] != r["gold_intent"]:
        kinds["wrong intent"] += 1
        mix_ups[(r["gold_intent"], r["predicted_intent"])] += 1
        print(f"wrong intent:       {request!r} (correct: {r['gold_intent']}, said:
{r['predicted_intent']})")

print("\ncounts:", dict(kinds))
print("most common mix-ups:", mix_ups.most_common(3))
```

The key lines:

Line	What it does
`r["status"] == "oos"`	the system refused the request
`r["predicted_intent"] != r["gold_intent"]`	the system answered with the wrong intent
`mix_ups.most_common(3)`	which intents get confused with which, most often

Run it:

```
python diy/c12_errors.py
```

What you should see (real output):

```
wrong intent:       'when is my meeting' (correct: calendar_set, said: calendar_query)
wrongly refused:    'when is bruno mars coming to sacramento' (correct: calendar_query)
wrong intent:       'clear event from calendar and refresh' (correct: calendar_remove, said:
calendar_query)
wrong intent:       'clear my schedule for today' (correct: calendar_remove, said:
calendar_query)
wrongly refused:    'please tell me how can i remove the item' (correct: lists_remove)
wrong intent:       'remind me to order more soap' (correct: lists_createoradd, said:
calendar_set)
wrongly refused:    'remove last played song' (correct: lists_remove)
wrongly refused:    "what is john's last name" (correct: email_querycontact)
wrongly refused:    'save this email in contact' (correct: email_addcontact)
wrongly refused:    'tell bob smith' (correct: email_sendemail)
wrongly refused:    'is this the correct area code for my boss' (correct: email_querycontact)
wrongly refused:    "please delete the contacts that i haven't spoken to in three of months"
(correct: email_querycontact)
wrongly refused:    'open the contact named john' (correct: email_querycontact)
wrongly refused:    'please open my contact list' (correct: email_querycontact)
wrong intent:       'i need to email susan and metion' (correct: email_sendemail, said:
email_addcontact)
wrongly refused:    'call raju phone number' (correct: email_querycontact)
answered off-topic: 'please add this song to my playlist favorites' -> lists_createoradd
answered off-topic: 'can you send a tweet to lif saying that their app crashed' ->
email_sendemail

counts: {'wrong intent': 5, 'wrongly refused': 11, 'answered an off-topic request': 2}
most common mix-ups: [(('calendar_remove', 'calendar_query'), 2), (('calendar_set',
'calendar_query'), 1), (('lists_createoradd', 'calendar_set'), 1)]
```

What to notice: It finds 5 wrong intents, 11 wrongly refused and 2 off-topic requests answered: exactly the intent-level counts in the project's error analysis. The project also sorts entity mistakes.

G13. Build it yourself: Robustness to messy typing (goes with C13)

Test robustness yourself: rewrite the requests messily and re-score the word-matching baseline.

The file: `diy\c13_robustness.py` (open it with `code diy\c13_robustness.py`):

```
"""C13 yourself: robustness. Make the requests messy and see whether the score holds.

Tests the word-matching baseline from c03 (with the same refusal rule), so it needs no model
and runs in seconds.
Run:  python diy/c13_robustness.py
"""

import random

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score

from helpers import load_jsonl

examples = load_jsonl("data/processed/P1/kshot.jsonl")
dev = [r for r in load_jsonl("data/processed/P1/dev.jsonl") if r["in_scope"]]


def add_typo(sentence, rng):
    """Swap two neighbouring letters in one word of 4 or more letters."""
    words = sentence.split()
    long_words = [i for i, w in enumerate(words) if len(w) >= 4]
    if not long_words:
        return sentence
    i = rng.choice(long_words)
    w = words[i]
    j = rng.randrange(len(w) - 1)
    words[i] = w[:j] + w[j + 1] + w[j] + w[j + 2:]
    return " ".join(words)


def add_filler(sentence, rng):
    return "hey, could you please " + sentence + ", thanks"


# Train the word-matching classifier once, as in c03.
vectorizer = TfidfVectorizer(ngram_range=(1, 2), sublinear_tf=True)
classifier = LogisticRegression(max_iter=2000, C=10.0, random_state=0)
classifier.fit(vectorizer.fit_transform([r["utt"] for r in examples]), [r["true_intent"] for
r in examples])
labels = sorted({r["true_intent"] for r in examples})
correct = [r["true_intent"] for r in dev]

for name, rewrite in [("clean", lambda s, rng: s), ("typos", add_typo),
                      ("CAPITALS", lambda s, rng: s.upper()), ("filler words", add_filler)]:
    rng = random.Random(0)
    messy = [rewrite(r["utt"], rng) for r in dev]
    vectors = vectorizer.transform(messy)
    predicted = classifier.predict(vectors)
    # Refuse when unsure, exactly as in c03: messy words make the classifier less sure.
    confidence = classifier.predict_proba(vectors).max(axis=1)
    predicted = ["REFUSED" if c < 0.35 else p for p, c in zip(predicted, confidence)]
    f1 = f1_score(correct, predicted, labels=labels, average="macro", zero_division=0)
    refused = predicted.count("REFUSED")
    print(f"{name:13} e.g. {messy[0]!r:50} macro-F1 {f1:.3f}  refused {refused} of
{len(dev)}")
```

The key lines:

Line	What it does
`def add_typo(sentence, rng)`	swaps two neighbouring letters in one long word
`s.upper()`	rewrites the request in capitals
`refused = predicted.count("REFUSED")`	how many requests it refused because it was unsure

Run it:

```
python diy/c13_robustness.py
```

What you should see (real output):

```
clean         e.g. 'turn the lights off please'                     macro-F1 0.771  refused
19 of 80
typos         e.g. 'turn the ligths off please'                     macro-F1 0.704  refused
27 of 80
CAPITALS      e.g. 'TURN THE LIGHTS OFF PLEASE'                     macro-F1 0.771  refused
19 of 80
filler words  e.g. 'hey, could you please turn the lights off please, thanks' macro-F1 0.699
refused 31 of 80
```

What to notice: Typos and filler words make it unsure, so it refuses more (27 and 31 of 80, against 19 when clean) and loses about 7 points. Capitals change nothing, because it lowercases everything. That is the same pattern as the project's robustness test.

G14. Build it yourself: Export and live demo (goes with C14)

Save a pipeline as a package yourself, protect it with a fingerprint, load it and use it.

The file: `diy\c14_export.py` (open it with `code diy\c14_export.py`):

```
"""C14 yourself: save a pipeline as a package, protect it with a fingerprint, load it and use
it.

Needs the Ollama app open for the last step.  Run:  python diy/c14_export.py
"""

import hashlib
import json
from pathlib import Path

from mini_pipeline import answer

# 1. Export: write the chosen design into a file.
package = {"project": "P1 home assistant", "examples": "fixed", "refusal": "model", "model":
"qwen2.5:3b"}
path = Path("diy/my_package.json")
path.write_text(json.dumps(package, indent=2), encoding="utf-8")

# 2. Fingerprint it: SHA-256 turns the file's exact bytes into a code. Change one letter
#    and the code changes completely.
fingerprint = hashlib.sha256(path.read_bytes()).hexdigest()
Path("diy/my_package.sha256").write_text(fingerprint, encoding="utf-8")
print("saved diy/my_package.json, fingerprint", fingerprint[:16], "...")


# 3. Load: check the fingerprint first; refuse a changed file.
def load(path):
    expected = Path("diy/my_package.sha256").read_text(encoding="utf-8")
    if hashlib.sha256(path.read_bytes()).hexdigest() != expected:
        raise ValueError(f"{path} was changed after it was saved: refusing to load it")
    return json.loads(path.read_text(encoding="utf-8"))


design = load(path)
print("loaded:", design)
for request in ["dim the kitchen lights", "who won the match yesterday"]:
    print(f"{request!r} -> {answer(request, examples=design['examples'],
refusal=design['refusal'])}")

# 4. Tamper with a copy and show it is refused.
tampered = Path("diy/my_package_tampered.json")
tampered.write_text(path.read_text(encoding="utf-8").replace("fixed", "similar"),
encoding="utf-8")
try:
    load(tampered)
except ValueError as error:
    print("tampered copy:", error)
```

The key lines:

Line	What it does
`hashlib.sha256(...).hexdigest()`	the fingerprint: change one letter of the file and it changes completely
`if ... != expected: raise ValueError`	refuses to load a file whose fingerprint no longer matches
`replace("fixed", "similar")`	tampers with a copy, to show it is refused

Run it:

```
python diy/c14_export.py
```

What you should see (real output):

```
saved diy/my_package.json, fingerprint 154cb8a7b6e28ded ...
loaded: {'project': 'P1 home assistant', 'examples': 'fixed', 'refusal': 'model', 'model':
'qwen2.5:3b'}
'dim the kitchen lights' -> iot_hue_lightdim
'who won the match yesterday' -> none_of_the_above
tampered copy: diy\my_package_tampered.json was changed after it was saved: refusing to load
it
```

What to notice: The saved package answers "dim the kitchen lights" correctly and refuses "who won the match yesterday", and the tampered copy is refused. The project's export works the same way, with more files.

G15. Build it yourself: Checking the results themselves (goes with C15)

Check a recorded result yourself by recomputing it from the saved answers.

The file: `diy\c15_check.py` (open it with `code diy\c15_check.py`):

```
"""C15 yourself: check a recorded result by recomputing it from the saved answers.

No model needed.  Run:  python diy/c15_check.py
                        python diy/c15_check.py 20261013_0110_P1_B1_0      (any run folder)
"""

import json
import sys
from pathlib import Path

from sklearn.metrics import accuracy_score, f1_score

from helpers import load_jsonl

folder = Path("runs") / sys.argv[1] if len(sys.argv) > 1 else
sorted(Path("runs").glob("*_P1_A2-selected_0"))[-1]
recorded = json.loads((folder / "metrics.json").read_text(encoding="utf-8"))
rows = [r for r in load_jsonl(folder / "predictions.jsonl") if r["gold_in_scope"]]

gold = [r["gold_intent"] for r in rows]
said = [r["predicted_intent"] if r["status"] == "ok" else "REFUSED" for r in rows]

my_f1 = f1_score(gold, said, labels=recorded["intent_label_set"], average="macro",
zero_division=0)
my_accuracy = accuracy_score(gold, said)
print("run:", folder.name)
print(f"intent macro-F1  recorded {recorded['primary']['intent_macro_f1']:.6f}   recomputed
{my_f1:.6f}")
print(f"accuracy         recorded {recorded['primary']['intent_accuracy_in_scope']:.6f}
recomputed {my_accuracy:.6f}")
same = abs(my_f1 - recorded["primary"]["intent_macro_f1"]) < 1e-9
print("-> the same" if same else "-> DIFFERENT")
```

The key lines:

Line	What it does
`json.loads(... metrics.json ...)`	the number the run recorded
`f1_score(gold, said, ...)`	the same number, recomputed with scikit-learn

Run it:

```
python diy/c15_check.py
```

What you should see (real output):

```
run: 20261024_1654_P1_A2-selected_0
intent macro-F1  recorded 0.974359   recomputed 0.974359
accuracy         recorded 0.950000   recomputed 0.950000
-> the same
```

What to notice: The recorded and recomputed values are identical to 6 decimal places. Give it any run folder name to check that run.

Practice with Part G

☐ Read `diy\c03_b0.py` and `diy\c04_b1.py` line by line using G3 and G4, and run them.
☐ Change one thing in each (for example the 0.35 cut-off in c03) and run again: explain why the score changed.
☐ If the teacher asks "could you build it yourself?": open `diy\c04_b1.py`, explain it, and run `python diy/c04_b1.py`. Your own 40-line pipeline scores 0.927, the same as B1.